

MACHINE LEARNING INTERVIEW PREPARATION

LLM Foundations: Top 53 Interview Questions & Answers

Transformer Blocks, Positional Embeddings, KV Cache & Serving Optimizations

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: Attention Math, GQA/MQA, KV Cache math, FlashAttention, PagedAttention, MoE routing, Deep Thinking TTC

Includes: Step-by-Step Numerical Hand Calculations, PyTorch Implementations, SVG/HTML Diagrams

LLM Foundations – Top 53 Interview Questions & Answers

1. Evolution of NLP (Q1–Q4)

Question 1: Why did Transformers replace RNNs and LSTMs?

[ESSENTIAL]

Conversational Answer

"I'd explain that the transition from RNNs to Transformers was driven by a fundamental hardware limitation. RNNs process text sequentially, meaning to compute the hidden state for the current word, you must wait for the hidden state of the previous word. This creates an $O(L)$ sequential dependency bottleneck that prevents you from parallelizing training across GPU cores. Transformers solved this by using self-attention, which allows the model to process all tokens in a sequence simultaneously. This unlocked parallel training, enabling us to scale models to billions of parameters on massive datasets."

Intuitive Example

- **Sequential vs. Parallel:** If you are processing "*The cat sat on the mat*", an LSTM must process "*The*", then "*cat*", then "*sat*" in order. A Transformer processes all six tokens at the exact same time through matrix multiplication, routing relationships via attention weights.

Key Interview Points

- **Recurrence Bottleneck:** RNNs require step-by-step sequential processing, limiting training speed.
 - **Self-Attention:** Processes the entire sequence in parallel, maximizing GPU compute occupancy.
 - **Constant Path Length:** Connects any two tokens with a path length of $O(1)$ operations, preventing vanishing gradients over long contexts.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Self-attention maps all inputs in parallel using Query, Key, and Value projections:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

In an LSTM, the distance between token 1 and token L is L steps, making gradient backpropagation through time (BPTT) prone to vanishing. In a Transformer, the distance is always a single matrix product, establishing a direct gradient path.

Production Perspective & Trade-offs

RNNs cannot saturate modern GPU Tensor Cores because sequential execution is memory-bandwidth bound (constantly loading weights to process a single token). Transformers pack sequences into large matrices, shifting the workload to be compute-bound during training, which yields orders-of-magnitude higher training throughput (TFLOPS).

Common Mistakes

1. Stating that Transformers are faster at inference. In autoregressive decoding, generation is still sequential (token-by-token) and memory-bandwidth bound.
2. Thinking Transformers have shorter training sequences than RNNs. They process much longer sequences due to parallelization.

COMMON FOLLOW-UP QUESTIONS

Q: If self-attention is parallel, how do we preserve word order?

We add Positional Encodings directly to the input embeddings so the model can distinguish word order.

Q: Does self-attention require more memory than LSTM?

Yes, self-attention memory scales quadratically ($O(L^2)$) with sequence length, whereas LSTMs scale linearly ($O(L)$).

One-Line Takeaway

Takeaway: Transformers replaced RNNs because self-attention eliminates the sequential training bottleneck, allowing efficient GPU parallelization and massive model scaling.

Question 2: What are the limitations of RNNs and Seq2Seq models?

[ESSENTIAL]

Conversational Answer

"The core issue with traditional Seq2Seq models is the 'information bottleneck'. In an encoder-decoder RNN, the encoder compresses the entire input sequence into a single, fixed-size hidden vector, which is then passed to the decoder. If you are translating a 100-word paragraph, forcing that entire

semantic meaning into a single vector of, say, 512 dimensions causes massive information loss. Additionally, sequential backpropagation through time makes LSTMs highly vulnerable to vanishing gradients over long sequences, meaning they forget early context."

Intuitive Example

- **Information Bottleneck:** Imagine reading a full page of a book and being forced to summarize it in exactly one word before passing it to a friend to translate. That is what a Seq2Seq model does without attention.

Key Interview Points

- **Information Bottleneck:** Compressing arbitrary-length inputs into a single fixed vector loses context.
 - **Vanishing Gradients:** RNN cells struggle to preserve gradient flow over steps greater than ~ 100 .
 - **Sequential Dependency:** Training speed is limited by the step-by-step nature of recurrence.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

In standard Seq2Seq, the context vector c passed to the decoder is simply the final encoder hidden state:

$$c = h_L$$

This mapping is constant. As sequence length $L \rightarrow \infty$, the representation density required to store context without degradation scales exponentially, causing severe loss of early-sequence information.

Production Perspective & Trade-offs

Because traditional Seq2Seq models are bound by $O(L)$ recurrence loops, they cannot scale to modern pre-training datasets (trillions of tokens). Training a 7B parameter RNN Seq2Seq model from scratch would take months longer than a Transformer equivalent due to poor GPU core saturation.

Common Mistakes

1. Conflating Seq2Seq with Encoder-Decoder. Seq2Seq is the task framework (mapping input to output sequence); Encoder-Decoder is a specific neural architecture that implements it.
2. Thinking LSTMs have no memory retention. They improve on vanilla RNNs but still decay over long contexts.

COMMON FOLLOW-UP QUESTIONS

Q: How did early Seq2Seq models try to resolve the bottleneck?

They reversed the input sequence during training to make early tokens closer to the decoder, and eventually introduced attention mechanisms.

Q: Does adding attention to Seq2Seq eliminate the recurrence bottleneck?

It resolves the information bottleneck, but the sequential recurrence bottleneck remains because the underlying layers are still RNNs.

One-Line Takeaway

Takeaway: RNN Seq2Seq models are limited by sequential training constraints and an information bottleneck that compresses long contexts into a single fixed-size vector.

Question 3: Compare Word2Vec, GloVe, and FastText.

[ESSENTIAL]

Conversational Answer

"I'd compare them based on how they construct embeddings. Word2Vec is a predictive, local window model—it trains a shallow network to predict a word given its neighbors. GloVe is a count-based global model—it factorizes the global co-occurrence matrix of the entire corpus. FastText improves on Word2Vec by treating words as bags of character n-grams. This is a huge production advantage because it allows FastText to generate embeddings for Out-of-Vocabulary words, like typos or slang, by looking at their subword parts."

Intuitive Example

- **Out-of-Vocabulary (OOV):** If the word "*subwavelength*" is missing from the vocabulary, Word2Vec and GloVe will return an error or `<unk>` token. FastText breaks it down into n-grams like `sub`, `wave`, `length` and sums their vectors to construct a highly accurate representation.

Key Interview Points

- **Word2Vec:** Local context window, predictive learning (Skip-gram/CBOW).
 - **GloVe:** Global co-occurrence matrix factorization, optimizing log co-occurrences.
 - **FastText:** Character n-grams, resolves OOV words, excellent for morphologically rich languages.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

FastText represents a word w as a sum of its character n -gram vectors v_g :

$$v_w = \sum_{g \in \mathcal{G}_w} v_g$$

This allows the representation space to share parameters across words with shared roots, prefixes, or suffixes.

Production Perspective & Trade-offs

All three models output **static embeddings**—each word has a single fixed vector. If a word has multiple meanings (e.g., "*bank*" as a river bank vs. money bank), these models average the vectors, causing semantic conflation. This requires modern contextual models (BERT/GPT) that compute embeddings dynamically.

Common Mistakes

1. Thinking GloVe is a deep neural network. It is a log-bilinear matrix factorization model based on count statistics.
2. Using Word2Vec for character-level tasks. Word2Vec operates strictly at the word level.

COMMON FOLLOW-UP QUESTIONS

Q: Why is CBOW faster than Skip-gram?

CBOW predicts one target word from multiple context words (averaging context vectors), while Skip-gram predicts multiple context words from one target, requiring more projection steps.

Q: How does FastText impact vocabulary memory footprint?

It increases memory usage because it must store vectors for millions of character n -grams in addition to the word vocabulary.

One-Line Takeaway

Takeaway: Word2Vec and GloVe generate static word-level embeddings, while FastText operates on character n -grams to handle Out-of-Vocabulary words.

Question 4: Why was the Attention mechanism introduced?

[ESSENTIAL]

Conversational Answer

"Attention was introduced to break the fixed-size vector bottleneck in Seq2Seq translation. Instead of forcing the decoder to translate a sentence based solely on the final hidden state of the encoder, attention allows the decoder to look back at all the encoder's hidden states at every step. It calculates a similarity score between what the decoder needs and what each encoder token represents, generating a weighted average context vector. This allows the model to align target words directly with source words dynamically."

Intuitive Example

- **Linguistic Alignment:** When a French-to-English Seq2Seq model generates the English word "apple" at step 3, the attention mechanism assigns a high weight (e.g. 0.95) to the French encoder token "pomme" at step 2, ensuring direct translation alignment.

Key Interview Points

- **Dynamic Context:** Decoder builds a unique context vector at each step by attending to all encoder states.
 - **Linguistic Alignment:** Directly maps source tokens to target tokens during generation.
 - **Gradient Shortcut:** Connects decoder states directly to early encoder hidden states, stabilizing gradients.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Bahdanau Attention computes alignment scores $e_{t,i}$ between decoder state s_{t-1} and encoder states h_i , normalizes them via softmax to get attention weights $\alpha_{t,i}$, and outputs context vector c_t :

$$c_t = \sum_{i=1}^L \alpha_{t,i} h_i$$

This replaces the static mapping $c = h_L$ with a dynamically routed representation.

Production Perspective & Trade-offs

Attention adds a computational layer: at every decoding step, we must calculate pairwise dot products against all encoder hidden states. This scales as $O(L_{\text{dec}} \times L_{\text{enc}})$, which is memory-intensive and

increases latency for long documents.

Common Mistakes

1. Thinking attention was invented for the Transformer. Attention was originally developed to improve RNN-based Seq2Seq models.
2. Believing attention weights represent actual causality. They represent correlation and routing patterns, not mathematical causal proofs.

COMMON FOLLOW-UP QUESTIONS

Q: What is the difference between global and local attention?

Global attention attends to all encoder tokens, while local attention restricts the search window to a small subset of positions to save compute.

Q: How does attention help with vanishing gradients?

It creates direct backpropagation pathways from the decoder to any encoder step, bypassing the long sequential recurrence path.

One-Line Takeaway

Takeaway: Attention was introduced to resolve the fixed-vector information bottleneck by allowing the decoder to dynamically query and retrieve context from all encoder states.

2. Transformer Architecture (Q5–Q12)

Question 5: Explain the Transformer architecture.

[ESSENTIAL]

Conversational Answer

"I'd describe the original Transformer as an Encoder-Decoder architecture built entirely on self-attention and feed-forward networks, completely avoiding recurrence. The encoder processes the input sequence bidirectionally to create contextual representations. The decoder takes these representations and causally generates the output sequence, token-by-token. Both parts use multi-head attention to capture relationships at different angles, positional encodings to track order, and residual connections with layer normalization to make training deep models stable."

Intuitive Example

- **Transformer Flow:** In a translation task, the encoder reads "*La vie est belle*" all at once, building a context map. The decoder starts with a start-of-sequence token and causally generates "*Life*", then queries the encoder context to generate "*is*", then "*beautiful*", ending with an end-of-sequence token.

Key Interview Points

- **No Recurrence:** Relies on self-attention to route information across positions in parallel.
 - **Encoder-Decoder:** Bidirectional context mapping combined with causal autoregressive generation.
 - **Residuals & Norms:** Essential topology components that stabilize gradient flow in deep stacks.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The architecture consists of stacked blocks.

Encoder Block:

$$\text{Output} = \text{LayerNorm}(x + \text{MultiHeadAttention}(x))$$

$$\text{Block_Out} = \text{LayerNorm}(\text{Output} + \text{FFN}(\text{Output}))$$

Decoder Block adds a masked self-attention layer followed by an encoder-decoder cross-attention layer:

$$\text{Cross_Attn} = \text{softmax}\left(\frac{Q_{\text{dec}}K_{\text{enc}}^T}{\sqrt{d_k}}\right)V_{\text{enc}}$$

Production Perspective & Trade-offs

The lack of recurrence makes training highly parallel, but storing layer activations for the backward pass requires massive GPU VRAM. In production, training pipelines use **gradient checkpointing** to recompute activations on the fly, trading compute for a smaller VRAM footprint.

Common Mistakes

1. Thinking modern LLMs use the original encoder-decoder layout. Most modern LLMs (GPT, Llama) are decoder-only.

2. Forgetting that positional encodings are added to the input; without them, the architecture is completely bag-of-words.

COMMON FOLLOW-UP QUESTIONS

Q: Why does the decoder need masked self-attention?

To prevent the model from looking at future tokens (cheating) during training on target sequences.

Q: What is the purpose of the projection layers after attention?

They project the concatenated head outputs back into the model's hidden dimension, mixing features across channels.

One-Line Takeaway

Takeaway: The Transformer is a recurrence-free architecture relying on self-attention, feed-forward networks, and residual connections to process sequence context in parallel.

Question 6: Compare Encoder-only, Decoder-only, and Encoder-Decoder models.

[ESSENTIAL]

Conversational Answer

"I'd contrast them by their attention masks and primary use cases. Encoder-only models, like BERT, use bidirectional attention, meaning every token looks at every other token. This is perfect for understanding tasks like sentiment analysis. Decoder-only models, like GPT and Llama, use causal attention—tokens can only look backward. This matches autoregressive generation. Encoder-Decoder models, like T5, combine both: a bidirectional encoder processes the source text, and a causal decoder generates the output. This is ideal for translation and summarization."

Intuitive Example

- **Attention Connectivity:**
 - **Encoder-only (BERT):** In "*The [mask] sat on the mat*", the model looks at both "*The*" and "*sat on the mat*" to predict "*cat*".
 - **Decoder-only (GPT):** In "*The cat sat*", the model can only look at "*The*" and "*cat*" to predict "*sat*"; it cannot see future words.

Key Interview Points

- **Encoder-only:** Bidirectional attention, representation focus, classification tasks.

- **Decoder-only:** Causal attention, autoregressive text generation focus.
 - **Encoder-Decoder:** Separate representation and generation blocks, sequence transformation tasks.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Attention Masks:**
 - Encoder: $A_{i,j}$ has no mask constraint.
 - Decoder: Attention weights $a_{i,j}$ are restricted by a lower-triangular causal mask:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

- Cross-Attention: $Q \in \mathbb{R}^{L_{\text{dec}} \times d_k}$ queries $K, V \in \mathbb{R}^{L_{\text{enc}} \times d_k}$ keys/values.

Production Perspective & Trade-offs

Decoder-only models are highly versatile and dominate general assistant tasks, but their inference is slow and memory-bandwidth bound. Encoder-only models run in a single forward pass, making them cheap and fast for classification or search pipelines.

Common Mistakes

1. Thinking encoder-only models are good at text generation. They cannot generate text autoregressively.
2. Forgetting that encoder-decoder models require separate KV Caches for both the self-attention and cross-attention blocks.

COMMON FOLLOW-UP QUESTIONS

Q: Why did decoder-only architectures win the scaling race?

They are simpler to scale, share parameters across context and generation, and handle few-shot prompting tasks naturally.

Q: Can you train a decoder-only model to do classification?

Yes, by passing prompt instructions and extracting classifications from the final generated token.

One-Line Takeaway

Takeaway: Encoder-only models are bidirectional for comprehension; decoder-only models are causal for generation; encoder-decoder models link both for translation.

Question 7: Why does GPT use only the decoder?

[ESSENTIAL]

Conversational Answer

"GPT uses a decoder-only architecture because it was designed for language modeling, which is next-token prediction. By removing the encoder, we simplify the model into a single homogeneous stack of causally masked attention blocks. This means we process the input prompt and the generated response in the exact same way, within the same layer representations. This homogeneity maximizes parameter sharing and scaling efficiency on GPUs."

Intuitive Example

- **Single Unified Flow:** Instead of having a source encoder and target decoder, GPT takes the prompt "Write a poem" and generated output "The sky..." as a single sequence ["Write", "a", "poem", "The", "sky", "..."], applying a single causal mask.

Key Interview Points

- **Homogeneity:** Single type of block (masked self-attention + FFN) simplifies scaling.
- **Parameter Sharing:** The prompt context and generated response share the exact same weights.
- **Inference Efficiency:** Eliminates cross-attention layers, saving compute parameter lookups.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

GPT models next-token probabilities causally:

$$P(X) = \prod_{i=1}^L P(x_i \mid x_1, \dots, x_{i-1})$$

By removing the separate encoder, the model does not require cross-attention layers ($Q_{\text{dec}}K_{\text{enc}}^T$), reducing block complexity.

Production Perspective & Trade-offs

Because the prompt context is ingested causally, early prompt tokens cannot attend to later prompt tokens during the prefill phase. This makes decoder-only models slightly less sample-efficient at understanding tasks compared to bidirectional encoders of the same parameter size.

Common Mistakes

1. Assuming GPT decoders are identical to original Transformer decoders. GPT decoders completely remove the cross-attention layer since there is no encoder.
2. Thinking causal masking is disabled during prompt ingestion. The mask is always active to keep training and inference consistent.

COMMON FOLLOW-UP QUESTIONS

Q: What is the main serving benefit of decoder-only models?

A single KV Cache pipeline manages the entire generation context, simplifying memory allocation engines.

Q: How does causal masking affect prompt processing speed?

The prompt is processed in parallel (prefill) in a single forward pass, but attention calculations are still masked lower-triangularly.

One-Line Takeaway

Takeaway: GPT is decoder-only because a single, causally masked stack of layers is highly homogeneous and optimal for scaling next-token prediction.

Question 8: Explain the role of Feed Forward Networks (FFNs).

[ESSENTIAL]

Conversational Answer

"I'd explain that while self-attention is great at routing information between tokens, it is a linear operation—essentially just weighted averages. If you only had self-attention, your model would struggle to learn complex, non-linear representations. The FFN layer is applied to each token position independently and in parallel. It uses non-linear activation functions to project the token features into a higher dimension and project them back, acting as a key-value store for factual knowledge and feature transformation."

Intuitive Example

- **Attention vs. FFN:** If you have the phrase "Apple stock" and "Apple fruit", attention routes the context word "stock" or "fruit" to "Apple". The FFN then processes this mixed vector at the "Apple" position, applying non-linear activation weights to map the token to the correct concept (finance vs. botany).

Key Interview Points

- **Non-Linear Capacity:** Introduces non-linear activations (GELU, SwiGLU) to learn complex representations.
 - **Position-wise Parallelism:** Processes each token position independently without cross-token interaction.
 - **Parameter Budget:** Consumes roughly 2/3 of a standard Transformer's parameter count.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Standard FFN projects input x of dimension d_{model} to intermediate dimension d_{ffn} (usually $4d_{\text{model}}$) using weight matrix W_1 , applies non-linearity, and projects back using W_2 :

$$\text{FFN}(x) = \text{Activation}(xW_1 + b_1)W_2 + b_2$$

Complexity scales linearly with sequence length: $O(L \cdot d_{\text{model}} \cdot d_{\text{ffn}})$.

Production Perspective & Trade-offs

Because FFNs are position-wise, they are compute-bound matrix multiplications (GEMMs). They achieve high hardware utilization on GPU Tensor Cores, making them highly FLOPS-efficient compared to memory-bound attention layers.

Common Mistakes

1. Thinking FFNs share information across different tokens. FFNs process each token position independently; cross-token interaction occurs only in the attention layers.
2. Assuming FFN parameters change across positions. The same weights W_1, W_2 are applied to all token positions within a layer.

COMMON FOLLOW-UP QUESTIONS

Q: Why does the intermediate dimension scale to $4d$?

Empirically, expanding the feature space by a factor of 4 provides the optimal capacity for non-linear feature maps.

Q: Can we run FFNs in parallel with Attention?

Yes, some architectures (like GPT-J or Falcon) compute FFN and Attention in parallel instead of sequentially to reduce latency.

One-Line Takeaway

Takeaway: FFNs process each token position independently using non-linear activations to project features, acting as the primary parameter store for representation learning.

Question 9: Why are Residual Connections important?

[ESSENTIAL]

Conversational Answer

"Residual connections are the reason we can train deep models. When you stack dozens of layers, gradients tend to vanish or explode as they are multiplied by weight matrices during backpropagation. A residual connection adds the layer's input directly to its output, like $x + F(x)$. This creates an uninterrupted 'gradient highway'. Gradients can flow directly back to the very first layer without being altered, preventing vanishing gradients and stabilizing training."

Intuitive Example

- **Identity Shortcut:** Imagine a game of telephone with 100 people. If each person rewrites the message, it gets corrupted. If instead each person only adds a small correction note to the original message, the core message survives to the end. That is a residual connection.

Key Interview Points

- **Additive Paths:** Implements $x_l = x_{l-1} + F(x_{l-1})$, keeping the identity path clean.
 - **Vanishing Gradient Solution:** Ensures gradients can backpropagate directly without vanishing.
 - **Identity Initialization:** Allows the model to start training by passing representations through unchanged.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

For a block with L layers, the state at layer L is:

$$x_L = x_0 + \sum_{i=1}^{L-1} F(x_i)$$

During backpropagation, the derivative contains a constant 1:

$$\frac{\partial x_L}{\partial x_0} = 1 + \sum_{i=1}^{L-1} \frac{\partial F(x_i)}{\partial x_0}$$

This constant 1 prevents the gradient from vanishing to zero, regardless of depth.

Production Perspective & Trade-offs

Residual connections require keeping the input activation x in GPU VRAM during the forward pass of the block $F(x)$, because we need x to perform the final addition. This increases the activation memory footprint, limiting batch sizes during training.

Common Mistakes

1. Thinking residual connections reduce computational complexity. They add no compute, but they increase VRAM allocation for activations.
2. Believing gradients flow only through the layer function. Gradients flow primarily through the identity shortcut path.

COMMON FOLLOW-UP QUESTIONS

Q: What is the impact of residual connections on feature scaling?

They cause the scale of activations to grow with depth, which is why normalization layers (LayerNorm/RMSNorm) are placed on the residual paths.

Q: Can we use residuals in convolutional neural networks?

Yes, this is the core innovation of ResNet, which allowed CNNs to scale past 100 layers.

One-Line Takeaway

Takeaway: Residual connections create additive shortcut pathways that allow gradients to

backpropagate directly through deep stacks without vanishing.

Question 10: LayerNorm vs RMSNorm.

[ESSENTIAL]

Conversational Answer

"LayerNorm stabilizes training by normalizing hidden states across the feature dimension, which requires calculating both the mean and the variance of the activations. RMSNorm simplifies this by arguing that we don't need the mean—only the scaling variance matters for training stability. So, RMSNorm normalizes by dividing by the Root Mean Square, skipping mean calculation. On GPUs, this is a big deal because it removes one reduction operation, saving memory access bandwidth and speeding up training."

Intuitive Example

- **Normalization Steps:**
 - **LayerNorm:** Shift all vector values so the mean is 0, then scale them so the variance is 1.
 - **RMSNorm:** Skip shifting the mean. Just scale the raw values directly so their root mean square is 1.

Key Interview Points

- **LayerNorm:** Calculates mean and variance; translates and scales inputs.
- **RMSNorm:** Calculates Root Mean Square only; scales inputs without mean centering.
- **GPU Throughput:** RMSNorm is faster because it reduces memory-bound reduction sweeps.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Comparison Table:**

Normalization Metric	LayerNorm	RMSNorm
Shift/Mean Centering	Yes ($x - \mu$)	No (raw x)
Scaling Metric	Standard Deviation (σ)	Root Mean Square (RMS)
Parameters	Learnable γ and β	Learnable γ only

Normalization Metric	LayerNorm	RMSNorm
GPU Reductions	2 passes (mean, variance)	1 pass (square sum)
Mathematical Formulation	$\frac{x-\mu}{\sigma} \odot \gamma + \beta$	$\frac{x}{\text{RMS}(x)} \odot \gamma$

Production Perspective & Trade-offs

Normalization is memory-bandwidth bound on GPUs (reading/writing to HBM). Removing the mean-centering step halves the memory reads needed for reduction, improving throughput by up to 10% on normalization-dense architectures.

Common Mistakes

- 1. Thinking RMSNorm has fewer scale parameters. Both use the same size learnable scale parameter γ ; RMSNorm simply removes the learnable bias parameter β .
- 2. Believing RMSNorm degrades model convergence. Empirical tests show convergence behavior is virtually identical.

COMMON FOLLOW-UP QUESTIONS

Q: Why is mean centering unnecessary?

Training stability comes primarily from scaling inputs to keep activations bounded, not from shifting the mean of the distribution.

Q: How does epsilon (ϵ) prevent division by zero in both?

It adds a tiny constant (e.g. $1e-5$) to the denominator to prevent mathematical instability when variance is zero.

One-Line Takeaway

Takeaway: RMSNorm replaces LayerNorm by normalizing inputs using only their root mean square, skipping mean calculation to save GPU memory bandwidth.

Question 11: Pre-LN vs Post-LN Transformers.

[ESSENTIAL]

Conversational Answer

"This comes down to where you place the normalization layer relative to the residual path. In Post-LN (the original design), normalization happens after the residual addition. This blocks the gradient highway, causing gradients to grow extremely large near the output layer, which requires careful learning rate warmups. In Pre-LN, we normalize the inputs *before* they enter the attention or FFN block. The residual path remains completely clean, which stabilizes gradient flow and allows us to train deep models without complex warmups."

Intuitive Example

- **Block Layouts:**
 - **Post-LN:** $x \rightarrow \text{Block} \rightarrow \text{Add} \rightarrow \text{LayerNorm} \rightarrow \text{Next Layer}$
 - **Pre-LN:** $x \rightarrow \text{LayerNorm} \rightarrow \text{Block} \rightarrow \text{Add} \rightarrow \text{Next Layer}$ (The identity path bypasses normalization).

Key Interview Points

- **Post-LN:** Normalizes output of addition; limits gradient flow, requires warmups.
 - **Pre-LN:** Normalizes input to blocks; keeps the residual path clean, highly stable.
 - **Modern Standard:** Virtually all modern LLMs (Llama, GPT-3) use Pre-LN.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Topologies:**

- Post-LN:

$$x_{l+1} = \text{LayerNorm}(x_l + F(x_l))$$

- Pre-LN:

$$x_{l+1} = x_l + F(\text{LayerNorm}(x_l))$$

- **Gradient Flow:** In Post-LN, gradients are scaled down by LayerNorm at each step back: $\frac{\partial x_l}{\partial x_{l-1}} \propto \frac{1}{\sigma_l}$. In Pre-LN, the identity derivative $\frac{\partial x_l}{\partial x_0} = 1 + \text{gradients}$ allows direct gradient flow.

Production Perspective & Trade-offs

Because activations accumulate along the unnormalized residual path in Pre-LN, the scale of hidden states grows with depth. We must add a final normalization block right before the output projection head to prevent activation explosion.

Common Mistakes

1. Thinking Pre-LN has different parameter counts. The block parameters are identical; only the layout configuration changes.
2. Assuming Pre-LN completely eliminates the need for learning rate schedules. You still need cosine decay, but you can skip or drastically shorten the warmup phase.

COMMON FOLLOW-UP QUESTIONS

Q: Why was Post-LN used in the original paper if Pre-LN is better?

Post-LN keeps the variance of activations constant across layers, which worked well for shallow (6-layer) models, but failed at scale.

Q: What is DeepNorm?

A newer normalization layout that combines elements of both to allow stable training of models with over 1000 layers.

One-Line Takeaway

Takeaway: Pre-LN normalizes activations prior to block execution, keeping the residual path clean to stabilize gradient flow in deep networks.

Question 12: Why do modern LLMs use GELU/SwiGLU instead of ReLU?

[ESSENTIAL]

Conversational Answer

"We avoid ReLU because of the 'dead neuron' problem. ReLU sets all negative values to zero, which means their gradient is exactly zero, and those weights stop updating. GELU and SwiGLU solve this by providing a smooth, non-monotonic curve with a small negative slope. This keeps gradients flowing even for negative inputs. SwiGLU also uses a gating mechanism where two parallel projections are multiplied element-wise. This provides higher capacity and sharper activation boundaries, which improves training efficiency."

Intuitive Example

- **Dead Neurons vs Gating:** If an input is -0.5 , ReLU outputs 0 (zero gradient). GELU outputs a tiny negative value, preserving information. SwiGLU uses the value to gate another projection, acting as a soft filter.

Key Interview Points

- **No Dead Neurons:** Bypasses ReLU's hard zero boundary to keep gradients flowing.
 - **Smoothness:** Continuous derivatives stabilize gradient updates.
 - **Gating Capacity:** Gated multiplication improves representation capacity.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Activations:**

- GELU:

$$\text{GELU}(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution.

- SwiGLU:

$$\text{SwiGLU}(x) = \text{Swish}_1(xW) \otimes (xV)$$

where $\text{Swish}_1(x) = x \cdot \sigma(x)$.

Production Perspective & Trade-offs

SwiGLU requires three projection matrices (W_{gate} , W_{value} , W_{down}) instead of two. To keep the parameter count equivalent to standard FFNs, we reduce the intermediate projection size: $d_{\text{ffn}} \approx \frac{8}{3}d_{\text{model}}$ (instead of $4d_{\text{model}}$).

Common Mistakes

1. Forgetting that SwiGLU requires more parameters if the intermediate dimension is not downscaled.
2. Thinking SwiGLU adds compute layers. It increases parameter footprint, but the computations remain highly efficient on GPU Tensor Cores.

COMMON FOLLOW-UP QUESTIONS

Q: What does the 'GLU' in SwiGLU stand for?

Gated Linear Unit, which is the general framework of using one linear projection to gate another.

Q: Is SwiGLU used in all modern models?

Yes, it is the standard in Llama-3, Gemma, Mistral, and PaLM.

One-Line Takeaway

Takeaway: Modern LLMs use SwiGLU because its smooth, gated projection avoids dead neurons and increases representational capacity.

3. Attention Mechanisms (Q13–Q22)

Question 13: Explain Self-Attention.

[ESSENTIAL]

Conversational Answer

"I'd describe self-attention as a routing mechanism that lets tokens in a sentence talk to each other to build context. Instead of treating words as static vectors, self-attention projects each token into Query, Key, and Value vectors. By taking the dot product of one token's Query with all tokens' Keys, we compute compatibility scores. We run these through softmax to get attention weights, and use them to take a weighted sum of the Values. This dynamic routing means each token gathers information from the words that are most relevant to it in that specific context."

Intuitive Example

- **Dynamic Information Routing:** In the sentence "*The bank of the river is muddy*," the token "*bank*" attends highly to "*river*" and "*muddy*". This context-aware attention routing updates the vector representation of "*bank*" to mean a land slope near water, not a financial institution.

Key Interview Points

- **Contextual Routing:** Tokens dynamically exchange information based on context, resolving ambiguity.
 - **Dynamic Weights:** Attention weights are calculated on the fly at runtime, not fixed after training.
 - **No Path Decay:** Connects any two tokens with a direct $O(1)$ relationship path.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Self-attention maps inputs X to Q, K, V via learnable projections W_q, W_k, W_v :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

The dot product QK^T generates an $L \times L$ similarity matrix. The softmax operation normalizes the row scores into a probability distribution.

Production Perspective & Trade-offs

The $L \times L$ similarity matrix creates an $O(L^2)$ computational and memory bottleneck. For a sequence length $L = 100,000$, just storing a single layer's attention scores requires massive GPU memory allocation, making context scaling the primary serving bottleneck.

Common Mistakes

1. Thinking self-attention itself has learnable parameters. The attention routing is parameter-free; the learnable weights reside in the projection matrices W_q, W_k, W_v, W_o .
2. Assuming self-attention operates sequentially. It is fully parallelized via matrix-matrix multiplication.

COMMON FOLLOW-UP QUESTIONS

Q: What is the computational complexity of self-attention?

It is $O(L^2 \cdot d)$ compute and $O(L^2)$ memory, where L is sequence length and d is head dimension.

Q: Why does self-attention use projection matrices?

They project input embeddings into distinct semantic subspaces, allowing query-key matches to capture different types of relationships.

One-Line Takeaway

Takeaway: Self-attention dynamically routes context-aware representations across all tokens in a sequence using query-key compatibility matching.

Question 14: What are Query, Key, and Value?

[ESSENTIAL]

Conversational Answer

"I like to use the analogy of a database lookup system. The **Query** represents the search term of the current token—what it is actively looking for. The **Key** represents the labels or index of all tokens in the sequence—what each token has to offer. The **Value** represents the actual content of the tokens. By multiplying the Query vector of a word with the Key vectors of all other words, we calculate how well they match. We use that match strength to retrieve a weighted mixture of their Value vectors."

Intuitive Example

- **Database Analogy:** In "*The chef cooked the soup*," the word "*cooked*" (Query: searching for *who* performed the action and *what* was created) matches the Key of "*chef*" (matching *subject*) and "*soup*" (matching *object*). It retrieves their respective Values to build a rich semantic representation of the cooking event.

Key Interview Points

- **Query (Q):** The semantic criteria the current token is searching for.
 - **Key (K):** The semantic categories the target tokens contain.
 - **Value (V):** The actual content extracted once a match is established.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Input vectors x_i are projected using weight matrices:

$$q_i = x_i W_q, \quad k_i = x_i W_k, \quad v_i = x_i W_v$$

The alignment score is the dot product:

$$\text{score}_{i,j} = q_i \cdot k_j$$

The output vector z_i at position i is:

$$z_i = \sum_j \text{softmax}(\text{score}_{i,:})_j v_j$$

Production Perspective & Trade-offs

In multi-head attention, the hidden dimension d_{model} is split across h heads, meaning the size of individual Q, K, V vectors is $d_k = d_{\text{model}}/h$. This keeps computation constant compared to single-head attention while increasing representation diversity.

Common Mistakes

1. Thinking Q, K, V are static lookups. They are computed dynamically at every layer from the preceding layer's output.
2. Confusing the Key and the Value. Keys are used to calculate the similarity scores; Values represent the actual content being mixed.

COMMON FOLLOW-UP QUESTIONS

Q: Can we set Q, K , and V to be identical to the input X ?

Yes, this is equivalent to raw dot-product attention, but adding projection matrices allows the model to project features into distinct semantic subspaces.

Q: Are Q, K , and V vectors of the same dimension?

Yes, they are typically projected into the same dimension d_k , though theoretically the Value dimension can differ from the Query-Key dimension.

One-Line Takeaway

Takeaway: Query acts as a semantic search term, Key as the database index, and Value as the content retrieved based on the query-key match.

Question 15: Why divide attention scores by $\sqrt{d}$?

[ESSENTIAL]

Conversational Answer

"We divide attention scores by $\sqrt{d_k}$ to prevent the dot products from growing extremely large in magnitude as the query/key dimension d_k increases. When vectors have high dimensionality, their dot products can easily blow up. Large dot products push the softmax function into regions with extremely small gradients—the flat saturation zones. This causes vanishing gradients during backpropagation, which halts training. Scaling by $\sqrt{d_k}$ preserves unit variance, stabilizing softmax and keeping gradients healthy."

Intuitive Example

- **Softmax Saturation:** If your dot products are $[100, 2, 1]$, softmax outputs $[1.0, 0.0, 0.0]$ —a hard one-hot distribution. The gradients at this point are practically zero. Scaling these scores down to, say, $[2.5, 0.05, 0.025]$ keeps the distribution soft, preserving gradient flow.

Key Interview Points

- **Variance Control:** Scaling keeps the variance of the dot product at 1.0.
 - **Softmax Saturation:** Prevents large values that push softmax to output near-one-hot distributions with zero gradients.
 - **Stable Training:** Essential for scaling hidden dimensions beyond 512.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Assuming components of q and k are independent random variables with mean 0 and variance 1.0, the variance of their dot product is:

$$\text{Var}(q \cdot k) = d_k$$

Dividing the dot product by $\sqrt{d_k}$ scales the variance back to 1.0, keeping the inputs to softmax within a stable range:

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = 1.0$$

Production Perspective & Trade-offs

Without scaling, gradients flowing through the softmax layers vanish, causing training runs to diverge early.

Common Mistakes

1. Saying division is used to normalize the scores between 0 and 1. Softmax handles the 0-1 normalization; $\sqrt{d_k}$ controls the variance of the inputs to softmax.
2. Thinking d is the sequence length. d_k is the projection dimension of the attention head.

COMMON FOLLOW-UP QUESTIONS

Q: Are there other scaling methods?

Yes, some architectures use alternative constants or query-key normalization layers, but $\sqrt{d_k}$ remains the standard.

Q: Does scaling affect inference speed?

No, scaling is a simple element-wise operation that adds negligible compute.

One-Line Takeaway

Takeaway: Scaling attention scores by the square root of the head dimension prevents softmax saturation and vanishing gradients during training.

Question 16: Explain Causal Masking.

[ESSENTIAL]

Conversational Answer

"Causal masking is the technique that makes a model autoregressive—meaning it can only look at past tokens, not future ones. During self-attention, we add a mask matrix to the query-key scores before applying softmax. The mask sets all future token indices (above the diagonal) to negative infinity. When we take the softmax, these negative infinity values evaluate to zero probability. This guarantees that when the model is predicting the next word, it cannot cheat by looking ahead."

Intuitive Example

- **Autoregressive Training:** When training on "*The cat sat on the mat*", when the model is at the token "*sat*", causal masking ensures it can only attend to "*The*", "*cat*", and "*sat*". The future tokens "*on*", "*the*", "*mat*" are masked out.

Key Interview Points

- **Autoregressive Constraint:** Prevents information leakage from target labels during training.
 - **Lower Triangular Matrix:** Attention weights $a_{i,j}$ are restricted to $j \leq i$.
 - **Negative Infinity Padding:** Scores for $j > i$ are set to $-\infty$ prior to softmax.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The causal mask M is added directly to the scaled query-key similarity matrix:

$$\text{Masked_Scores} = \frac{QK^T}{\sqrt{d_k}} + M$$

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Softmax evaluated on $-\infty$ yields 0, forcing the attention weights for future positions to evaluate to 0.0.

Production Perspective & Trade-offs

During training, causal masking allows parallel processing of the entire sequence (we calculate all sequence losses in a single forward pass). During inference, we generate one token at a time, making the causal mask redundant beyond the active sequence limit, which is optimized using KV Caches.

Common Mistakes

1. Thinking causal masking is applied after softmax. The mask must be applied before softmax (by setting scores to $-\infty$), otherwise weights would not sum to 1.0 over valid tokens.
2. Using causal masking in encoders. Encoders should look bidirectionally; causal masking is reserved for autoregressive decoders.

COMMON FOLLOW-UP QUESTIONS

Q: Can we use causal masking in encoders?

Technically yes, but it defeats the encoder's purpose of extracting full bidirectional context.

Q: How does causal masking affect prompt ingestion (prefill)?

The prompt is processed in parallel in a single forward pass, but attention calculations are still masked lower-triangularly.

One-Line Takeaway

Takeaway: Causal masking blocks information flow from future tokens by adding negative infinity to future scores before the softmax operation.

Question 17: Why is Self-Attention $O(n^2)$?

[ESSENTIAL]

Conversational Answer

"Self-attention scales quadratically because every word in a sentence must compare itself to every other word. If you have a sequence of length L , you have to compute dot products between all L queries and all L keys. This constructs an $L \times L$ similarity matrix. As sequence lengths grow—say, from 1,000 tokens to 100,000 tokens—the size of this matrix grows by a factor of 10,000, creating a massive compute and VRAM memory bottleneck."

Intuitive Example

- **Quadratic Scaling:** If a 1,000-token prompt requires a similarity matrix of 1×10^6 elements, a 10,000-token prompt requires a matrix of 1×10^8 elements. Storing and calculating this matrix scales quadratically.

Key Interview Points

- **Pairwise Interactions:** Every token queries every key, creating L^2 dot products.
 - **Matrix Dimension:** The similarity matrix has dimensions $[L, L]$.
 - **Memory Boundary:** Storing the float attention matrix for backpropagation is a major bottleneck.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The compute cost of calculating QK^T is:

$$\text{Compute}_{\text{Attention}} = 2 \cdot L^2 \cdot d_{\text{model}} \text{ FLOPs}$$

where L is sequence length. The memory complexity required to store the attention score activations is:

$$\text{Memory}_{\text{Attention}} = O(L^2 \cdot h) \text{ bytes}$$

where h is the number of heads. As sequence length L grows larger than the hidden dimension d_{model} , the quadratic term dominates.

Production Perspective & Trade-offs

For $L = 100,000$ and $d = 4096$, the raw attention matrix size is 10^9 elements. At float16 precision, just storing a single layer's attention scores requires **20 GB** of GPU VRAM, making long context inference extremely memory-intensive.

Common Mistakes

1. Stating that FFN layers scale quadratically. FFNs process tokens independently, meaning they scale linearly ($O(L)$) with sequence length.
2. Thinking that only compute is quadratic. The VRAM memory footprint for storing activations is also $O(L^2)$, which is often the harder bottleneck.

COMMON FOLLOW-UP QUESTIONS

Q: What are linear attention alternatives?

Architectures like state-space models (Mamba) or sparse attention variants (Sliding Window Attention) try to achieve $O(L)$ scaling.

Q: How does FlashAttention reduce this bottleneck?

FlashAttention does not reduce FLOPs, but it reduces the memory bottleneck by avoiding writing the $L \times L$ matrix to GPU High Bandwidth Memory.

One-Line Takeaway

Takeaway: Self-attention is quadratic because calculating pairwise similarity scores across all positions creates an $L \times L$ matrix bottleneck.

Question 18: Explain Multi-Head Attention.

[ESSENTIAL]

Conversational Answer

"Instead of running a single attention mechanism over the entire model dimension d_{model} , Multi-Head Attention splits the Queries, Keys, and Values into h smaller subspaces. We run attention on all of these heads in parallel. Once they are done, we concatenate the outputs and project them back to the original model dimension. This allows the model to focus on different types of relationships simultaneously—for example, one head might track grammatical structure, while another resolves pronouns."

Intuitive Example

- **Subspace Diversity:** In the sentence "The cat sat on the mat because it was tired," one attention head might connect "it" to "cat" (coreference resolution), while another head connects "sat" to "mat" (prepositional mapping). Multi-head attention allows both to be processed at the same time.

Key Interview Points

- **Subspace Diversity:** Allows different heads to focus on different syntactic or semantic relationships.
 - **Constant Compute:** Splitting the vectors means MHA compute cost is identical to single-head attention of dimension d_{model} .
 - **Output Projection:** Concatenated head outputs are mixed via weight matrix W_o .
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Multi-Head Attention projects Q, K, V into h heads, each with dimension $d_k = d_{\text{model}}/h$:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

where projection weights have shape:

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$$

Production Perspective & Trade-offs

Splitting shapes to `[B, h, L, d_k]` allows batched matrix multiplications using efficient GPU libraries.

Common Mistakes

1. Thinking MHA performs h times more computations than single-head attention. Because individual head dimensions are scaled down ($d_k = d_{\text{model}}/h$), total FLOPs remain equivalent.
2. Assuming all heads capture the same features. Each head is initialized randomly, causing them to specialize in different linguistic patterns.

COMMON FOLLOW-UP QUESTIONS

Q: Does increasing head count h always improve performance?

No, if h is too large, the head dimension d_k becomes too small to capture rich features, degrading model capacity.

Q: How do we combine the outputs of different heads?

We concatenate the h output matrices of shape $[B, L, d_k]$ back into $[B, L, d_{\text{model}}]$ and multiply by W^O .

One-Line Takeaway

Takeaway: Multi-Head Attention splits the hidden dimension into parallel heads to capture diverse syntactic and semantic relationships simultaneously.

Question 19: What does each attention head learn?

[ESSENTIAL]

Conversational Answer

"Attention heads specialize in different linguistic patterns. Some heads are local, attending only to the next or previous token. Others are syntactic, mapping relationships like verbs to their direct objects or prepositions. Some are semantic, matching coreferences—like linking 'she' to 'Alice'. We also see 'induction heads', which learn to copy patterns like 'if A follows B, then predict B after A', which is the key mechanism behind in-context learning."

Intuitive Example

- **Induction Pattern:** If a head sees the pattern "*Harry Potter... Harry [blank]*", it attends back to the first instance of "*Harry*" and copies the following token "*Potter*". This allows the model to replicate complex patterns.

Key Interview Points

- **Local Heads:** Attend to adjacent tokens.
 - **Syntactic Heads:** Map grammatical structures (e.g., subject-verb relationships).
 - **Semantic/Reference Heads:** Resolve coreferences (e.g., matching "he" to "John").
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

We can analyze head diversity by checking the entropy of attention weight distributions:

$$H(a) = - \sum_j a_j \log a_j$$

A low entropy head behaves like a pointer (focusing on a single token, e.g., the period or subject). A high entropy head acts as a broad context accumulator (spreading weights across all tokens).

Production Perspective & Trade-offs

Because some heads learn redundant mappings or have high similarity, serving systems can prune inactive or redundant heads after training without significant loss in accuracy.

Common Mistakes

1. Assuming we explicitly guide heads during training. Head specialization emerges naturally from random initialization and gradient descent.
2. Thinking all heads are equally important. Pruning studies show that many heads can be removed at inference time without impacting output quality.

COMMON FOLLOW-UP QUESTIONS

Q: What are "induction heads"?

Specialized heads that learn patterns like ``[A][B] ... [A] -> [B]``, which are essential for copying text and in-context learning.

Q: How do we visualize head attention weights?

Using tools like BertViz, which plot the attention weight matrix as lines connecting tokens.

One-Line Takeaway

Takeaway: Attention heads specialize naturally in local, syntactic, and semantic relationships, as well as pattern-copying induction mechanisms.

Question 20: Self-Attention vs Cross-Attention.

[ESSENTIAL]

Conversational Answer

"The difference is where the input vectors come from. In self-attention, the Queries, Keys, and Values all originate from the same sequence. This allows tokens within that sequence to interact. In cross-attention (used in encoder-decoder models), the Queries come from the decoder (the target sequence), but the Keys and Values come from the encoder (the source sequence). This allows the decoder to search and retrieve context from the encoder's output."

Intuitive Example

- **Mapping Inputs:**
 - **Self-Attention:** In "*The cat sat*," "*cat*" attends to "*The*" and "*sat*".
 - **Cross-Attention:** In translating French to English, when the decoder generates "*cat*", it queries the encoder's representations of "*Le chat*" to align the translation.

Key Interview Points

- **Self-Attention:** Q, K, V come from the same source tensor (X).
 - **Cross-Attention:** Q comes from the decoder state (H_{dec}); K, V come from the encoder outputs (H_{enc}).
 - **Mapping Alignment:** Cross-attention aligns target generation tokens to input prompt tokens.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Formulation Contrast:**

- Self-Attention:

$$Q = K = V = X \implies \text{Attention}(XW_q, XW_k, XW_v)$$

- Cross-Attention:

$$Q = X_{\text{dec}}W_q, \quad K = Y_{\text{enc}}W_k, \quad V = Y_{\text{enc}}W_v$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Production Perspective & Trade-offs

In self-attention, keys and values change with every generated token, requiring step-by-step caching updates. In cross-attention, the encoder outputs are static once computed, meaning the cross-attention keys and values do not change during decoding, saving compute memory updates.

Common Mistakes

1. Thinking cross-attention uses causal masking. Cross-attention does not use causal masking because it maps to static encoder states that are fully computed in advance.
2. Assuming decoder-only models use cross-attention. Decoder-only models have no encoder, so they use self-attention exclusively.

COMMON FOLLOW-UP QUESTIONS

Q: Why do decoder-only models omit cross-attention?

Because they process input prompts and generated text in a single causal sequence, removing the need for a separate encoder-decoder boundary.

Q: Which is more computationally expensive?

Self-attention is typically more expensive because the sequence length grows with each step, whereas cross-attention keys/values are static in size.

One-Line Takeaway

Takeaway: Self-attention routes information within a single sequence, while cross-attention links the decoder to the encoder's static representations.

Question 21: What is FlashAttention, and why is it faster?

[ESSENTIAL]

Conversational Answer

"FlashAttention is faster because it addresses the memory-bandwidth bottleneck, not the compute bottleneck. Standard attention computes the $L \times L$ attention matrix and constantly writes and reads it to and from the slow High Bandwidth Memory (HBM) on the GPU. FlashAttention uses a technique called tiling: it loads inputs in small blocks (tiles) into the fast, local GPU Shared Memory (SRAM), computes the attention output block-by-block using an online softmax algorithm, and updates the output without ever writing the massive intermediate $L \times L$ matrix to HBM."

Intuitive Example

- **SRAM Tiling:** Imagine you have a massive spreadsheet. Instead of constantly loading the entire sheet from your hard drive (HBM) to your RAM (SRAM) for every calculation, you load it block-by-block, perform all calculations on that block, write the result, and discard the block.

Key Interview Points

- **IO-Awareness:** Optimizes GPU memory hierarchy access rather than reducing FLOP counts.
 - **SRAM vs HBM:** Loads data into local SRAM (fast, ~19 TB/s) instead of global HBM (slow, ~2 TB/s).
 - **Online Softmax:** Computes softmax block-by-block using scaling factors, avoiding the need to store the full matrix.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

FlashAttention reduces HBM memory access scaling from quadratic $O(L^2)$ to linear $O(L)$. The online softmax computes running maximums m and denominators s to scale updates dynamically when merging blocks:

$$m^{\text{new}} = \max(m^{(1)}, m^{(2)})$$

$$s^{\text{new}} = s^{(1)} e^{m^{(1)} - m^{\text{new}}} + s^{(2)} e^{m^{(2)} - m^{\text{new}}}$$

Production Perspective & Trade-offs

FlashAttention yields a 2x to 4x speedup in training and inference without any approximation or loss in model accuracy. It enables context lengths to scale to 32k or higher on standard GPU hardware.

Common Mistakes

1. Thinking FlashAttention is an approximate attention method (like local or sparse attention). FlashAttention is mathematically identical to standard attention, yielding the exact same outputs.
2. Believing FlashAttention reduces the number of operations (FLOPs). It actually computes the same (or slightly more) FLOPs, but runs faster by optimizing memory access.

COMMON FOLLOW-UP QUESTIONS

Q: What is the main difference in FlashAttention-2?

It improves work partitioning across GPU thread blocks (warps), achieving up to 70% of peak theoretical GPU compute efficiency.

Q: Does FlashAttention work on all hardware?

It requires GPUs with shared memory architectures (like NVIDIA Ampere A100 or Hopper H100) and specific CUDA kernel implementations.

One-Line Takeaway

Takeaway: FlashAttention accelerates attention by loading tokens block-by-block into local SRAM, computing online softmax without writing the large similarity matrix to HBM.

Question 22: Why does FlashAttention perform more FLOPs in the backward pass yet runs faster?

[ESSENTIAL]

Conversational Answer

"This is a classic memory vs. compute trade-off. Standard attention writes the $L \times L$ attention matrix to HBM during the forward pass so it can be reused in the backward pass. FlashAttention does not store this matrix to save memory. In the backward pass, it has to recompute the attention scores block-by-block on the fly. Even though this recomputation adds more operations (FLOPs), doing calculations in fast SRAM registers is much faster than loading a massive matrix from slow HBM, resulting in a net speedup."

Intuitive Example

- **Recomputation Trade-off:** Imagine you need a calculation sheet. Instead of paying to store it in a warehouse miles away and paying to ship it back (HBM access), you quickly re-write the calculations on a whiteboard in your room (SRAM recomputation) whenever you need them.

Key Interview Points

- **Recomputation:** Recalculates the forward attention weights during the backward pass.
 - **Memory Bound vs Compute Bound:** GPU memory bandwidth is the bottleneck, not raw processing speed (FLOPS).
 - **SRAM Tiling:** Recomputed values are processed directly in local registers.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Modern GPUs (A100, H100) have extremely high compute capacity (~312 TFLOPS) but relatively slow memory bandwidth (~2 TB/s). FlashAttention leverages this by trading cheap FLOPs to save expensive memory access. It reduces the activation VRAM scaling from quadratic $O(L^2)$ to linear $O(L)$, allowing larger batch sizes.

Production Perspective & Trade-offs

By not storing the intermediate $L \times L$ attention matrix, FlashAttention reduces activation memory scaling from quadratic $O(L^2)$ to linear $O(L)$, allowing models to train with much larger batch sizes and sequence lengths.

Common Mistakes

1. Thinking FlashAttention performs more FLOPs in the forward pass. In the forward pass, it performs the same FLOPs but reduces memory IO; only the backward pass adds FLOPs due to recomputation.
2. Believing this recomputation degrades model accuracy. The recomputation is mathematically exact, so gradients remain identical.

COMMON FOLLOW-UP QUESTIONS

Q: Is this trade-off related to gradient checkpointing?

Yes, it is a localized, hardware-specific application of gradient checkpointing applied directly inside the attention CUDA kernel.

Q: How much VRAM does this save during training?

It reduces activation memory by up to 10x, which is often the main factor determining the maximum sequence length a GPU can train.

One-Line Takeaway

Takeaway: FlashAttention recomputes the forward attention matrix in fast SRAM during the backward pass, trading cheap FLOPs to avoid slow HBM reads.

4. Positional Encoding (Q23–Q25)

Question 23: Why do Transformers need positional information?

[ESSENTIAL]

Conversational Answer

"I'd explain that self-attention is mathematically permutation invariant. This means that if you shuffle the order of the words in a sentence, the attention values remain exactly the same—they just get routed to the new shuffled indices. Without positional information, a Transformer treats a sentence as a bag of words, processing the phrase 'not good' and 'good not' identically. We need to inject positional signals to break this symmetry so the model can learn grammar, syntax, and order."

Intuitive Example

- **Permutation Invariance:** Without positional encodings, the sequences "*cat eats fish*" and "*fish eats cat*" produce identical token representations at the attention layers. Adding positional signals gives each word unique spatial coordinates.

Key Interview Points

- **Permutation Invariance:** Self-attention processes sets of vectors, ignoring coordinate order.
 - **Symmetry Breaking:** Positional encodings add position-dependent values to token vectors.
 - **Grammatical Ordering:** Essential for learning syntax and semantic meaning derived from word order.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

If P is a permutation matrix that swaps token rows, self-attention on permuted input PX yields:

$$\text{Attention}(PQ, PK, PV) = P \cdot \text{Attention}(Q, K, V)$$

The representations themselves do not change based on sequence location; they are merely shuffled. Adding positional vectors PE directly to the input embeddings $X_{\text{out}} = X + PE$ breaks this permutation symmetry.

Production Perspective & Trade-offs

Absolute positional encodings are added once at the input embedding layer, which is computationally cheap. However, they limit the model to a fixed context length during training; the model has no

parameters or references to process position indices larger than those seen during training.

Common Mistakes

1. Thinking positional encodings are learned weights added to the attention matrix directly. Standard absolute encodings are added once to input token vectors.
2. Assuming the model automatically learns word order through causal masking. Causal masking restricts attention to past tokens but does not assign coordinate index values to those past positions.

COMMON FOLLOW-UP QUESTIONS

Q: Why not use a sequential RNN input layer to capture order?

This would introduce the sequential recurrence bottleneck we want to avoid, preventing parallel training.

Q: What happens if we omit positional encodings during training?

The model behaves like a bag-of-words model, losing structural syntax and performing poorly on downstream language generation.

One-Line Takeaway

Takeaway: Transformers require positional encodings because self-attention is permutation invariant and cannot distinguish token order without explicit spatial coordinates.

Question 24: Explain Rotary Positional Embeddings (RoPE).

[ESSENTIAL]

Conversational Answer

"Rotary Position Embedding, or RoPE, is a relative positional encoding method. Instead of adding fixed vectors to token embeddings at the input layer, RoPE takes the Query and Key vectors at *each* attention layer, splits their dimensions into 2D slices, and rotates them by an angle proportional to the token's position. Because of the trigonometry involved, when you compute the dot product of a Query at position m and a Key at position n , the result naturally depends only on their relative distance $m - n$."

Intuitive Example

- **Vector Rotation:** Imagine a clock face. If the Query token is at position 2 (rotated by 2θ) and the Key token is at position 5 (rotated by 5θ), their dot product evaluates the difference between their angles (3θ), preserving their relative distance.

Key Interview Points

- **Geometric Rotation:** Applies 2D rotations to Query and Key vector slices.
 - **Relative Distance:** Dot product naturally preserves relative token offsets ($m - n$).
 - **No Parameters:** Rotation is mathematically pre-computed, adding zero learnable parameters.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

For a 2D Query vector slice $q = (q_1 \quad q_2)^T$ at position m , we multiply by the rotation matrix:

$$R_{\Theta, m}^2 \begin{pmatrix} q_1 \\ q_2 \end{pmatrix} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} q_1 \\ q_2 \end{pmatrix}$$

Because rotation is orthogonal ($R^T R = I$), the dot product of rotated query and key satisfies:

$$(R_m q)^T (R_n k) = q^T R_{n-m} k$$

This mathematically preserves their relative distance $n - m$.

Production Perspective & Trade-offs

RoPE rotation matrices are large. Rather than performing full matrix multiplications, we compute rotations using element-wise vector operations:

$$x_{\text{rotated}} = x \odot \cos(m\Theta) + \text{rotate_half}(x) \odot \sin(m\Theta)$$

where `rotate_half` swaps and negates vector halves, saving memory bandwidth.

Common Mistakes

1. Thinking RoPE is applied to Value (V) vectors. RoPE is applied only to Queries (Q) and Keys (K) to calculate attention weights; Values (V) are left unrotated.
2. Believing RoPE is a static additive embedding. It is a multiplicative rotation applied at every attention layer.

COMMON FOLLOW-UP QUESTIONS

Q: What is NTK-aware scaling for RoPE?

A method that scales the base frequency θ instead of sequence length, allowing models to process context windows larger than those seen during training without retraining.

Q: Why is RoPE split into 2D slices?

Because rotation is geometrically defined in a 2D plane; high-dimensional vectors are split into independent 2D planes to rotate.

One-Line Takeaway

Takeaway: RoPE rotates Query and Key vector slices in 2D to mathematically build relative distance dependencies directly into attention dot products.

Question 25: Compare Sinusoidal Encoding, Learned Embeddings, RoPE, and ALiBi.

[ESSENTIAL]

Conversational Answer

"I'd compare them based on how they encode positions and how well they extrapolate to longer sequences. Learned embeddings are simple but fail to handle sequences longer than the training limit. Sinusoidal encoding uses static trig curves and extrapolates poorly. RoPE rotates Query and Key vectors to encode relative distance geometrically, and is the modern standard because it behaves stably. ALiBi bypasses vector addition entirely; it simply adds a negative penalty directly to attention scores based on distance, offering the strongest length extrapolation."

Intuitive Example

- **Extrapolation:** If a model is trained on a 2,048-token context:
 - **Learned:** Cannot run at 4,096 tokens (indices past 2,048 have no weights).
 - **RoPE:** Extrapolates to 4,096 tokens using frequency scaling (e.g. NTK-aware).
 - **ALiBi:** Handles 8,000+ tokens out of the box due to constant linear distance penalties.

Key Interview Points

- **Learned:** Simple lookup; zero extrapolation capability.
- **RoPE:** Multiplicative rotation at each layer; standard in modern LLMs (Llama-3, Gemma).

- **ALiBi:** Linear bias penalty added to attention scores; zero training parameters, robust extrapolation.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Comparison Table:**

Feature	Learned Embeddings	Sinusoidal Encoding	RoPE	ALiBi
Type	Absolute, Additive	Absolute, Additive	Relative, Multiplicative	Relative, Bias Penalty
Params	Learned weights	Static trig functions	Static trig rotations	Static linear bias
Extrapolate	None	Poor	Moderate (needs scaling)	Excellent
Application	BERT, GPT-2	Original Transformer	Llama-3, Mistral, Gemma	MPT, BLOOM

In ALiBi, attention scores are computed as:

$$\text{Score}(q_i, k_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}} - m \cdot |i - j|$$

where m is a constant slope hyperparameter determined per attention head.

Production Perspective & Trade-offs

ALiBi is simple and fast, but its linear penalty can over-penalize long-range dependencies, causing the model to ignore distant tokens even when they are highly relevant. RoPE, combined with NTK frequency scaling, has become the dominant industry standard due to its flexibility.

Common Mistakes

1. Thinking RoPE adds parameters. Like sinusoidal encoding, RoPE's rotation frequencies are mathematically pre-computed and do not add parameters.
2. Assuming ALiBi modifies query-key projection weights. It adds a static bias matrix directly to the attention logits.

COMMON FOLLOW-UP QUESTIONS

Q: Why does ALiBi extrapolate so well?

Because the negative penalty scales linearly and is independent of parameter updates, meaning the model's attention scores degrade predictably at long distances without exploding.

Q: What is RoFormer?

The original model architecture that introduced RoPE.

One-Line Takeaway

Takeaway: Learned embeddings do not extrapolate; RoPE rotates vector slices for relative representation; ALiBi adds a linear distance penalty to attention scores.

5. Tokenization & Embeddings (Q26–Q30)

Question 26: Explain Byte Pair Encoding (BPE).

[ESSENTIAL]

Conversational Answer

"BPE is a subword tokenization algorithm. It balances vocabulary size and sequence length. We start by splitting text into individual character tokens. Then, we count co-occurrences and merge the most frequent adjacent token pairs in our corpus into a new token. We repeat this process—merging pairs like 't' and 'h' into 'th'—until we reach our target vocabulary size. BPE is a standard because it compresses common words into single tokens while maintaining character-level fallback for rare words, avoiding out-of-vocabulary errors."

Intuitive Example

- **BPE Merges:** If the training corpus contains "*hug*" and "*pug*" frequently, the tokenizer initially sees characters `h`, `u`, `g`, `p`. It counts that `u-g` is a frequent pair and merges it to `ug`. Then, it merges `h` and `ug` into `hug`, and `p` and `ug` into `pug`.

Key Interview Points

- **Subword Tokenization:** Balances word-level vocabulary limits with character-level sequence bloat.
 - **Iterative Merges:** Greedily merges the most frequent adjacent byte/character pairs.
 - **Vocabulary Budget:** The vocabulary size is a hyperparameter (typically 32k to 128k).
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

BPE operates statistically, not neural-network based:

$$\text{PairScore}(t_a, t_b) = \text{Count}(t_a, t_b)$$

At each step, we identify:

$$(t_a^*, t_b^*) = \arg \max_{t_a, t_b} \text{Count}(t_a, t_b)$$

We merge this pair globally across the corpus, add it to our vocabulary, and repeat.

Production Perspective & Trade-offs

A larger vocabulary size reduces sequence token lengths (which saves compute during inference), but increases the embedding layer's parameter footprint and memory usage.

Common Mistakes

1. Believing BPE requires training a neural network. BPE is a deterministic statistical algorithm based purely on co-occurrence frequency counts in text.
2. Thinking BPE runs on characters only. Modern LLMs use Byte-level BPE, which operates on raw UTF-8 bytes to handle foreign characters and emojis.

COMMON FOLLOW-UP QUESTIONS

Q: How does BPE handle unseen words at test time?

It decomposes the unseen word into its constituent subwords or characters. In Byte-level BPE, any word can be represented using base UTF-8 bytes.

Q: Why is vocabulary size a critical hyperparameter?

It determines the tradeoff between embedding memory footprint (larger vocab = more VRAM) and sequence lengths (larger vocab = fewer tokens per sentence = faster inference).

One-Line Takeaway

Takeaway: BPE is an iterative statistical algorithm that merges the most frequent adjacent character/subword pairs to build a compressed vocabulary.

Question 27: Compare BPE, WordPiece, and SentencePiece.

[ESSENTIAL]

Conversational Answer

"BPE merges character pairs based on raw co-occurrence counts. WordPiece is similar but selects merges that maximize the likelihood of the training corpus under a unigram language model. SentencePiece is the modern standard for LLMs because it treats whitespace as a regular character—using a special underscore symbol. This means it doesn't need language-specific pre-tokenizers and can process raw byte streams directly, making it highly robust for multilingual models."

Intuitive Example

- **Whitespace Preservation:**
 - **BPE:** Splits text on spaces first, throwing them away, turning "hello world" to ["hello", "world"] .
 - **SentencePiece:** Treats spaces as `␣`, yielding ["_hello", "_world"] . Concatenating these tokens reconstructs the raw text perfectly.

Key Interview Points

- **WordPiece:** Likelihood-based merges (maximizing mutual information); used in BERT.
- **SentencePiece:** Byte-stream input, whitespace-aware, pre-tokenizer free; used in Llama.
- **Byte-Fallback:** Translates unknown characters to hex bytes instead of outputting `<unk>` .

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Comparison Table:**

Feature	BPE	WordPiece	SentencePiece
Merge Criterion	Frequency count	Likelihood / Entropy	Frequency or Unigram
Spaces	Split / Dropped	Split / Dropped	Treated as active char (<code>␣</code>)
Pre-Tokenizer	Required	Required	Not required (raw bytes)
Application	GPT-2, GPT-4	BERT	Llama-3, Gemma, T5

Production Perspective & Trade-offs

In production, byte-fallback is crucial. If a user types a rare emoji or character, SentencePiece converts it to 4 byte tokens (`<0xF0><0x9F>...`). While this increases sequence length, it prevents system crashes or loss of semantic information.

Common Mistakes

1. Thinking WordPiece and BPE are identical. BPE merges based on raw frequency counts; WordPiece merges by maximizing statistical likelihood under a probabilistic model.
2. Believing SentencePiece is an entirely new tokenization algorithm. SentencePiece is a software wrapper that implements BPE or Unigram on raw byte streams without pre-splitting text.

COMMON FOLLOW-UP QUESTIONS

Q: Why do modern decoders use SentencePiece over BPE?

Because it is language-agnostic and does not require complex regular expressions or rules to pre-tokenize text.

Q: What is byte-fallback?

Converting any out-of-vocabulary character directly into its constituent UTF-8 byte tokens rather than mapping to an unknown `` token.

One-Line Takeaway

Takeaway: BPE uses frequency merges; WordPiece maximizes likelihood; SentencePiece processes raw byte streams, preserving whitespace as a character.

Question 28: Why does token count matter?

[ESSENTIAL]

Conversational Answer

"Token count is the main driver of computational cost, latency, and billing in LLMs. Because self-attention is quadratic, processing more tokens increases GPU memory and compute requirements. Also, tokenization efficiency dictates your context capacity. A poor tokenizer will represent the same text using more tokens, which consumes more of your context window and increases API cost."

Intuitive Example

- **Tokenizer Compression:**

- **English:** "Hello, how are you?" → 5 tokens in Llama-3 (1:1 compression).
- **Non-English (e.g. Hindi):** "नमस्ते, आप कैसे हैं?" → 12 tokens in older tokenizers for the same phrase, making non-English processing 2.4x more expensive.

Key Interview Points

- **Attention Scaling:** $O(L^2)$ attention compute bottlenecks.
 - **VRAM Caching:** KV cache sizes scale linearly with generated tokens.
 - **Tokenizer Compression:** A larger vocabulary reduces token count per sentence.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Sequence processing cost scales quadratically with sequence length L :

$$\text{Complexity} = O(L^2)$$

For API billing, costs are computed based on prompt length L_{prompt} and generation length L_{gen} :

$$\text{Cost} = c_1 \cdot L_{\text{prompt}} + c_2 \cdot L_{\text{gen}}$$

Production Perspective & Trade-offs

Increasing vocabulary size (e.g., from 32k to 128k in Llama-3) reduces sequence length by 15-20%, speeding up inference and reducing cost, but increases the static model parameter size in VRAM by hundreds of megabytes.

Common Mistakes

1. Assuming 1 token is always equivalent to 1 word. In English, 1 token is typically ~0.75 words, but this ratio changes significantly across languages.
2. Thinking characters and tokens scale linearly in all tokenizers. Emojis and code sequences scale differently based on tokenizer design.

COMMON FOLLOW-UP QUESTIONS

Q: Why are API providers charging per token instead of per character?

Because GPU compute blocks process tokens as embedding lookups, making token count the direct metric of floating-point operations (FLOPs) performed.

Q: How do you measure tokenizer efficiency?

By calculating the average bytes-per-token ratio on a representative corpus; higher is better.

One-Line Takeaway

Takeaway: Token count dictates computational complexity and VRAM consumption due to quadratic self-attention and linear KV cache memory scaling.

Question 29: What are contextual embeddings?

[ESSENTIAL]

Conversational Answer

"Unlike static embeddings where a word always has the same vector, contextual embeddings are generated dynamically at runtime based on the surrounding words. The model uses self-attention to mix information from neighbor tokens. This means the word 'bank' in 'river bank' and 'investment bank' will have two completely different vector representations that capture the specific context of the sentence."

Intuitive Example

- **Polysemy Resolution:** If you process "*He deposited money in the bank*" and "*The river bank was steep*", a contextual model like BERT outputs two distinct vectors for "*bank*", resolving the ambiguity dynamically.

Key Interview Points

- **Dynamic Context:** Embeddings are computed dynamically at runtime using attention weights.
- **Polysemy Resolution:** Solves ambiguity by encoding surrounding semantic cues.
- **Layer-wise Depth:** Representations evolve through layers, moving from syntactic features to semantic abstractions.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Contextual hidden states $H^{(l)}$ at layer l are computed from the preceding layer's states:

$$H^{(l)} = \text{LayerNorm}(H^{(l-1)} + \text{Attention}(H^{(l-1)}))$$

The vector $h_i^{(l)}$ at position i contains information from all other positions j , making it a contextualized representation.

Production Perspective & Trade-offs

Generating contextual embeddings requires running the full forward pass of the model, which is much slower and more compute-intensive than retrieving static vectors from a lookup table.

Common Mistakes

1. Confusing static input embeddings with contextual hidden state embeddings. Input embeddings are static lookups; hidden states are contextualized.
2. Thinking contextual embeddings are static after training. They change with every new prompt configuration.

COMMON FOLLOW-UP QUESTIONS

Q: Which layers of an LLM provide the best contextual embeddings for search?

Typically, intermediate layers provide the best semantic balance. The final layer's activations are often highly specialized for the training objective (like next-token prediction).

Q: How do we extract sentence embeddings from contextual word embeddings?

By averaging the token representations (mean pooling) or extracting the vector of the special classification token (like `[CLS]` in BERT).

One-Line Takeaway

Takeaway: Contextual embeddings generate dynamic, neighborhood-aware representations for tokens at runtime by passing them through self-attention layers.

Question 30: Why is cosine similarity commonly used for embeddings?

[ESSENTIAL]

Conversational Answer

"We use cosine similarity because it measures the angle between vectors, ignoring their magnitude. In high-dimensional spaces, a longer document will have larger embedding vectors simply because it has more words, even if its topic is identical to a shorter document. By looking only at the angle, cosine similarity isolates semantic alignment, mapping the similarity between -1.0 and $+1.0$ regardless of scale."

Intuitive Example

- **Magnitude Invariance:** A 5-page document about cooking and a 1-sentence recipe will have vectors with vastly different lengths. However, their angle in semantic space is narrow. Cosine

similarity captures this shared topic, whereas Euclidean distance would show them as highly distant.

Key Interview Points

- **Magnitude Invariance:** Insensitive to the length of the input texts.
 - **Angle Focus:** Focuses strictly on directional alignment in semantic space.
 - **Compute Efficiency:** On normalized vectors, cosine similarity is a simple dot product.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Cosine similarity computes the cosine of the angle between vectors A and B :

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

If we normalize vectors to unit length ($\|A\| = 1$, $\|B\| = 1$) during index generation, the formula simplifies to a dot product:

$$\text{CosineSimilarity}(A, B) = A \cdot B$$

Production Perspective & Trade-offs

Computing cosine similarity on unnormalized vectors requires repeatedly calculating square roots for magnitudes ($\|A\|$), which degrades search speed. Unit-normalization is a mandatory pre-processing step for low-latency retrieval.

Common Mistakes

1. Believing cosine similarity is always superior to Euclidean distance. If vectors are unit-normalized, cosine similarity and Euclidean distance are monotonically equivalent, but Cosine similarity is faster to calculate.
2. Assuming cosine similarity maps to a true metric space distance. It does not satisfy the triangle inequality, which is why vector databases map it to angular distance for indexing.

COMMON FOLLOW-UP QUESTIONS

Q: When would you use Dot Product instead of Cosine Similarity?

When vector magnitude contains useful semantic signal (e.g. popular items in recommendation systems having larger embeddings).

Q: What is the relationship between Cosine Similarity and L2 Distance?

For unit-normalized vectors, $L2_Distance(A, B) = \sqrt{2(1 - \text{CosineSimilarity}(A, B))}$.

One-Line Takeaway

Takeaway: Cosine similarity measures directional alignment in embedding space while ignoring vector magnitudes, isolating semantic content from document length.

6. Training & Fine-Tuning (Q31–Q36)

Question 31: How is GPT trained using next-token prediction?

[ESSENTIAL]

Conversational Answer

"GPT is trained using self-supervised learning on raw text. We feed the model a sequence of tokens, pass them through the decoder layers, and use a classification head at the output to predict the probability distribution for the next token. We compare this prediction to the actual next word in the text using cross-entropy loss and backpropagate to update weights. Because we use causal masking, we can calculate the loss for *every* position in the sequence in a single forward pass, making training highly parallel and efficient."

Intuitive Example

- **Next-Token Loop:** For the text "*Deep learning is fast*", the model performs three predictions in one step:
 1. Input: ["Deep"] → Predict: "learning"
 2. Input: ["Deep", "learning"] → Predict: "is"
 3. Input: ["Deep", "learning", "is"] → Predict: "fast"

Key Interview Points

- **Self-Supervised:** Labels are derived directly from the text sequence itself.

- **Autoregressive Loss:** Minimizes the negative log-likelihood of predicting $P(x_t \mid x_{<t})$.
 - **Causal Parallelism:** Uses causal masking to process all sequence targets simultaneously during training.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

We minimize the cross-entropy loss over a sequence of length L :

$$\mathcal{L} = - \sum_{i=1}^{L-1} \log P(x_{i+1} \mid x_1, \dots, x_i)$$

For each token position, the final hidden state h_i is multiplied by the transposed input embedding matrix (weight tying) to generate raw vocabulary logits z_i , which are normalized via softmax.

Production Perspective & Trade-offs

The final classification layer has shape `[d_model, V]`. For Llama-3 ($d_{\text{model}} = 8192$, $V = 128k$), this projection contains **1 Billion parameters** (~2 GB at fp16), making it a major VRAM and memory-bandwidth consumer during training and inference.

Common Mistakes

1. Believing next-token prediction is run sequentially during training. Sequential generation only occurs at inference; training processes all tokens in parallel via causal masking.
2. Thinking the prompt doesn't calculate loss during pre-training. In pre-training, loss is calculated at every single token position.

COMMON FOLLOW-UP QUESTIONS

Q: What is weight tying?

Sharing the weight parameters between the input token embedding layer and the final output vocabulary classification layer to save VRAM.

Q: How does vocabulary size affect memory?

A larger vocabulary size increases the embedding and classification layer sizes, consuming more GPU memory but decreasing average tokenized sequence length.

One-Line Takeaway

Takeaway: GPT is trained by projecting hidden states to vocabulary logits and using causal masking to compute next-token cross-entropy loss for all positions in parallel.

Question 32: Explain Cross-Entropy Loss and Teacher Forcing.

[ESSENTIAL]

Conversational Answer

"Cross-entropy loss measures how far our predicted token probabilities are from the true next token. **Teacher Forcing** is the training strategy where we always feed the true ground-truth token as the input to the next step, even if the model predicted the wrong word in the previous step. This keeps the model on track and allows us to calculate losses in parallel. If we didn't use Teacher Forcing, one early mistake would derail the entire sequence, making training slow and unstable."

Intuitive Example

- **Teacher Guidance:** If the model is learning "*The cat sat on the mat*" and predicts "*The dog*" at step 2, Teacher Forcing ignores the mistake and still inputs "*cat*" at step 3. This ensures the model learns the correct context path instead of compounding its own errors.

Key Interview Points

- **Cross-Entropy:** Minimizes divergence between prediction probability and target index.
- **Teacher Forcing:** Feeds target tokens x_t to the input regardless of prediction y_{t-1} .
- **Exposure Bias:** Training uses ground-truth inputs, but inference uses model predictions, creating a mismatch (exposure bias).

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Cross-entropy loss for a single step with target index y is:

$$\text{Loss} = -\log P(\hat{y} = y)$$

Without Teacher Forcing, training would require sequential generation: we would have to sample token y_{t-1} at step $t - 1$ and feed it as input at step t . This prevents parallel GPU matrix multiplications, rendering training scaling impossible.

Production Perspective & Trade-offs

Because of the discrepancy between training (always getting true tokens) and inference (getting its own generated tokens), errors accumulate during decoding. This is known as **exposure bias** and is typically mitigated using reinforcement learning alignment (DPO/RLHF).

Common Mistakes

1. Thinking Teacher Forcing is used at inference. During inference, the ground truth is unavailable, so the model must feed its own predictions back as inputs.
2. Believing cross-entropy measures semantic similarity. It only measures exact word index matches; predicting a close synonym still yields high loss if it doesn't match the target vocabulary index.

COMMON FOLLOW-UP QUESTIONS

Q: How does exposure bias impact long text generation?

Small errors in early tokens compound over time, leading to drift, repetitive loops, or hallucinations.

Q: What is Scheduled Sampling?

A training curriculum that slowly transitions from feeding ground-truth tokens to model-generated tokens to bridge the gap between training and inference.

One-Line Takeaway

Takeaway: Teacher Forcing feeds ground-truth tokens as inputs at every training step to stabilize gradient updates and enable parallel sequence calculations.

Question 33: Pretraining vs Fine-Tuning vs Instruction Tuning.

[ESSENTIAL]

Conversational Answer

"I'd contrast them by scale and objective. **Pre-training** is training from scratch on trillions of tokens of raw text using next-token prediction, which teaches the model grammar, reasoning, and world facts. **Fine-tuning** is adapting that model to a specific task, like classification or sentiment, using small labeled datasets. **Instruction Tuning** is a form of fine-tuning where we train the pre-trained model on formatted prompt-response pairs to teach it how to behave as a helpful assistant that follows user instructions."

Intuitive Example

Training Steps:

- **Pre-training:** Reading the entire internet to learn language.
- **Instruction Tuning:** Training the model to respond to prompts like "Explain gravity" with structured explanations instead of simply writing the next sentence of an article.
- **Fine-tuning:** Training the model to classify emails as spam or not spam.

Key Interview Points

- **Pre-training:** High cost, self-supervised next-token prediction, builds base representation capability.
- **Instruction Tuning:** Medium cost, aligns base models to behave as conversational assistants (SFT).
- **Fine-tuning:** Low cost, adapts weights for narrow, specialized tasks (often using PEFT/LoRA).

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Pre-training:**

$$\text{Loss} = -\log P(x_i \mid x_{<i})$$

- **Instruction Tuning:** Let prompt be P and response be R . We causally mask the prompt tokens so the loss is evaluated only on the response:

$$\text{Loss} = -\sum_{t \in R} \log P(r_t \mid P, r_{<t})$$

Production Perspective & Trade-offs

Adapting models through full fine-tuning can trigger **catastrophic forgetting**, where the model loses general capabilities in exchange for task-specific performance. We mitigate this by mixing a small percentage of pre-training data into the tuning dataset.

Common Mistakes

1. Thinking instruction tuning adds factual knowledge. Factual knowledge is primarily encoded during pre-training; instruction tuning teaches the model how to retrieve and format that knowledge.
2. Believing fine-tuning always requires updating all model weights. Modern pipelines use Parameter-Efficient Fine-Tuning (PEFT/LoRA) to update less than 1% of parameters.

COMMON FOLLOW-UP QUESTIONS

Q: What is Catastrophic Forgetting?

A phenomenon where a neural network loses previously learned skills when trained on a new, narrow task distribution.

Q: Why are base models hard to use in chat applications?

Because they act as document completers; typing a question often prompts them to write another question rather than an answer.

One-Line Takeaway

Takeaway: Pre-training builds general language representations; instruction tuning aligns models to follow prompts; fine-tuning adapts them to specific tasks.

Question 34: What is Supervised Fine-Tuning (SFT)?

[ESSENTIAL]

Conversational Answer

"Supervised Fine-Tuning, or SFT, is the first step in converting a raw base model into a chatbot. We compile a dataset of high-quality prompt-response pairs, like 'User: Write a poem. Assistant: The wind blows...'. We pass the sequence through the model, but we mask the user's prompt during backpropagation. This ensures we only calculate losses and update weights based on the assistant's response tokens, training the model to emulate helpful conversational answers."

Intuitive Example

- **Prompt Masking:** If the sequence is `["Q:", "What", "is", "2+2?", "A:", "4"]`, SFT masks the logits for `["Q:", "What", "is", "2+2?", "A:"]` and only evaluates the loss on the output token `"4"`. The model is not penalized for failing to predict the user's question.

Key Interview Points

- **Prompt Masking:** Gradient updates are restricted to target response tokens only.
 - **Alignment Foundation:** Teaches the model the structure of dialogue and formatting (like JSON).
 - **Quality over Quantity:** A few thousand high-quality instructions outperform millions of noisy inputs.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Given prompt tokens $P = [p_1, \dots, p_k]$ and response tokens $R = [r_1, \dots, r_m]$, the SFT loss is:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{m} \sum_{j=1}^m \log P(r_j \mid p_1, \dots, p_k, r_1, \dots, r_{j-1})$$

The loss at positions p_1 to p_k is explicitly zeroed out during backpropagation to prevent parameter capacity from being wasted on copying prompt styles.

Production Perspective & Trade-offs

SFT model performance is highly sensitive to training data formatting. If your deployment system uses a different chat template (e.g. ChatML vs. Alpaca styles) than the one used during SFT, the model's output quality and safety boundaries can break in production.

Common Mistakes

1. Computing gradients on the user's prompt. This forces the model to learn to generate the user's questions, wasting capacity and degrading conversational quality.
2. Thinking SFT requires millions of examples. Modern instruction tuning focuses on high-quality, human-curated datasets (often fewer than 10,000 examples).

COMMON FOLLOW-UP QUESTIONS

Q: What is the "alignment tax"?

A decrease in raw academic or logical benchmark performance that occurs when aligning a model to be safe and conversational.

Q: How do we choose the prompt format?

Using standardized templates like Jinja2 to map system prompts, user queries, and assistant responses to specific token boundaries.

One-Line Takeaway

Takeaway: SFT adapts base models to chat formats by evaluating next-token cross-entropy loss exclusively on target response tokens, masking out prompt inputs.

Question 35: What are RLHF and Direct Preference Optimization (DPO)?

[ESSENTIAL]

Conversational Answer

"RLHF and DPO are alignment techniques that make models helpful, honest, and harmless. In traditional RLHF, we train a separate Reward Model on human preference choices between two responses. Then, we use reinforcement learning (like PPO) to optimize the LLM to maximize this reward, using a KL-divergence constraint to keep it from outputting gibberish. DPO is a modern alternative that skips the reward model entirely. It mathematically proves that we can optimize the policy directly on preference pairs using a simple binary cross-entropy loss, making training faster, simpler, and much more stable."

Intuitive Example

- **Preference Learning:** If a user prefers response A ("*Here is the code...*") over response B ("*I cannot help...*"), DPO increases the probability of generating A while decreasing the probability of B directly, without needing a separate reward model or RL loop.

Key Interview Points

- **RLHF:** Multi-model setup (Actor, Critic, Reward, Reference) optimized via PPO; unstable and VRAM-heavy.
 - **DPO:** Single cross-entropy loss on preference pairs; eliminates reward models and RL loops.
 - **KL-Divergence Penalty:** Prevents the active model from drifting too far from the base model, avoiding reward-hacking.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

DPO optimizes policy π_θ directly on preference pairs (x, y_w, y_l) where y_w is preferred (winning) and y_l is dispreferred (losing) given prompt x :

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

The hyperparameter β controls the strength of the KL-divergence constraint against the reference model π_{ref} .

Production Perspective & Trade-offs

RLHF (PPO) requires hosting 4 separate models in VRAM (Actor, Critic, Reward, Reference) during training, making it extremely memory-intensive. DPO only requires loading the active training policy π_θ and the frozen reference model π_{ref} , halving VRAM requirements.

Common Mistakes

1. Thinking DPO is unsupervised. DPO requires labeled preference datasets containing chosen and rejected response pairs for every prompt.
2. Believing DPO completely eliminates reward models. It bypasses training one explicitly, but the mathematical loss function still implicitly models human reward.

COMMON FOLLOW-UP QUESTIONS

Q: What is reward hacking?

A behavior where an RL agent optimizes for high reward scores by outputting nonsensical text (e.g. repeating keywords) that exploits reward model gaps.

Q: Why is PPO unstable?

Because RL policy updates have high variance; updates that are too large can destroy the model's language capability.

One-Line Takeaway

Takeaway: DPO simplifies RLHF alignment by mathematically reformulating preference optimization into a direct binary cross-entropy loss on prompt-response pairs.

Question 36: Why does scaling more data often outperform scaling more parameters (Chinchilla Scaling Laws)?

[ESSENTIAL]

Conversational Answer

"Chinchilla scaling laws corrected early beliefs about model size. Previously, people scaled parameter counts quickly while leaving training data sizes relatively small. The Chinchilla paper proved that for a compute-optimal budget, parameters and training tokens should scale in equal proportion. However, in production, serving costs depend strictly on the number of parameters. So, it is highly rational to 'over-train' a smaller model on significantly more tokens. It costs more during training, but it pays off in production by lowering VRAM and latency serving costs."

Intuitive Example

- **Inference Amortization:**
 - **Kaplan Optimal:** Train a 70B parameter model on 1.4 Trillion tokens.
 - **Production Optimal (Llama-3):** Train an 8B parameter model on 15 Trillion tokens. The 8B model is much cheaper to host and run, but achieves similar quality.

Key Interview Points

- **Kaplan vs. Chinchilla:** Kaplan overstated parameter scaling; Chinchilla showed parameters and tokens should scale equally ($D \approx 20N$).
 - **Inference Amortization:** Spending more compute during training to shrink parameter count lowers long-term serving costs.
 - **Compute Budget:** Total floating-point operations scale as $C \approx 6ND$.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The loss $L(N, D)$ as a function of parameters N and tokens D is modeled as:

$$L(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E$$

Chinchilla empirical scaling showed $\alpha \approx 0.34$ and $\beta \approx 0.28$. For a compute budget $C \approx 6ND$, this dictates that both should scale almost equally.

Production Perspective & Trade-offs

Serving a model to millions of users means inference compute dominates total lifecycle costs. Over-training smaller models past the Chinchilla limit is standard practice because the high training compute cost is quickly amortized over serving queries.

Common Mistakes

1. Thinking Chinchilla laws dictate how to get the highest quality model regardless of compute. Chinchilla laws strictly focus on maximizing training quality *under a fixed compute budget*.
2. Assuming token over-training has no limit. Eventually the model reaches its representation capacity ceiling, where training on more tokens yields diminishing returns.

COMMON FOLLOW-UP QUESTIONS

Q: How many tokens are compute-optimal for a 7B model according to Chinchilla?

Approximately 140 Billion tokens. Modern 7B models are trained on 15+ Trillion tokens, making them highly over-trained but cheap to serve.

Q: How does this affect VRAM budgets?

Over-training smaller models allows us to fit high-quality weights into single-GPU VRAM limits (like fitting 8B models into 16 GB).

One-Line Takeaway

Takeaway: Chinchilla laws show parameters and data should scale equally, but production serving costs favor over-training smaller models on massive token sets.

7. Inference & Text Generation (Q37–Q43)

Question 37: Explain autoregressive text generation.

[ESSENTIAL]

Conversational Answer

"Autoregressive generation is the process of generating text token-by-token. In each step, we feed the entire input sequence into the model, run a forward pass, and sample one next token. We then append this new token to our input sequence and repeat the process. Because each step requires a full forward pass through the network just to generate one word, decoding is highly sequential and memory-bandwidth bound on GPUs."

Intuitive Example

- **Step-by-Step Loop:**
 - Step 1: Input: "Translate: Cat" → Output: "Chat"
 - Step 2: Input: "Translate: Cat Chat" → Output: "<eos>" (generation ends).

Key Interview Points

- **Token-by-Token:** Sequential execution loops; generation is $O(L)$ forward passes.
- **Sequence Appending:** The generated output at step t becomes part of the input for step $t + 1$.
- **Bandwidth Bound:** Each decoding step requires reloading all model parameters to process a single token.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The joint probability of a generated sequence is the product of conditional probabilities:

$$P(x_1, x_2, \dots, x_L \mid \text{Prompt}) = \prod_{t=1}^L P(x_t \mid \text{Prompt}, x_1, \dots, x_{t-1})$$

At step t , the decoder computes logits $z_t = f(\text{Prompt}, x_{1:t-1})$. We sample token $x_t \sim \text{sample}(\text{softmax}(z_t))$.

Production Perspective & Trade-offs

Because each token requires loading the entire model's weights from HBM to SRAM to perform simple vector-matrix math, decoding GPU utilization is extremely low. We optimize this using **continuous batching** and **quantization** to maximize serving throughput.

Common Mistakes

1. Thinking the prompt ingestion (prefill) is also autoregressive. The prompt is ingested in parallel in a single forward pass; only subsequent generation is autoregressive.
2. Assuming decoding latency is compute-bound. Latency is limited by memory bandwidth (speed of loading weights to SRAM), not raw FLOPS.

COMMON FOLLOW-UP QUESTIONS

Q: How does speculative decoding speed this up?

By using a tiny draft model to generate candidate tokens quickly, and validating them in a single parallel forward pass of the larger target model.

Q: What triggers the end of generation?

The model generates a special End-of-Sequence (``) token, or the sequence length reaches context limits.

One-Line Takeaway

Takeaway: Autoregressive generation generates text sequentially by repeatedly appending the predicted next token to the input for the next forward pass.

Question 38: Compare Greedy Search, Beam Search, Top-k, Top-p, and Temperature sampling.

[ESSENTIAL]

Conversational Answer

"I'd group these into deterministic search algorithms and stochastic sampling methods. Greedy search is simple—it just takes the single most probable token at each step, which often leads to repetitive loops. Beam search tracks multiple paths in parallel, keeping the top- N most likely sequences; it is common in translation but rare in chat because it lacks creativity. Top- K and Top- P introduce controlled randomness: Top- K restricts sampling to the top K choices, while Top- P dynamically selects a subset based on cumulative probability. Temperature scales the logits before softmax: low temperature (< 1.0) makes outputs structured and predictable, while high temperature (> 1.0) increases creativity and variety."

Intuitive Example

- **Sampling Choices:** If the token probabilities are "apple" (40%), "banana" (30%), "cat" (20%), "dog" (10%):
 - **Greedy:** Always selects "apple".
 - **Top-K** ($K = 2$): Restricts selection to "apple" and "banana".
 - **Top-P** ($P = 0.8$): Selects from "apple", "banana", and "cat" (sum is 90% $\geq$ 80%).

Key Interview Points

- **Greedy:** Fast, deterministic, prone to repetitive output loops.
- **Beam Search:** Evaluates multiple pathways; compute-intensive, common in translation.
- **Top-K / Top-P:** Restricts selection space to static count K or dynamic cumulative probability P .
- **Temperature:** Alters the entropy of the output probability distribution by scaling logits.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Comparison Table:**

Method	Type	Parameters	VRAM Cost	Output Style
Greedy Search	Deterministic Search	None	Low	Repetitive, robotic

Method	Type	Parameters	VRAM Cost	Output Style
Beam Search	Deterministic Search	Beam width B	High ($B \times$ cache)	Structured, factual
Top-K Sampling	Stochastic Sampling	Token count K	Low	Controlled variety
Top-P Sampling	Stochastic Sampling	Probability threshold p	Low	Dynamic, natural
Temperature	Logit Scaling	Scaler τ	Low	$\tau < 1$ structured; $\tau > 1$ creative

Temperature scaling modifies logits z_i :

$$p_i = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}$$

Production Perspective & Trade-offs

Top-P is preferred in production because it adjusts the sampling pool dynamically. If the model is confident (one token has 99%), the pool shrinks to 1 token. If the model is uncertain, the pool expands, avoiding the text corruption that static Top-K limits can cause.

Common Mistakes

1. Thinking temperature alters the model parameters. Temperature is applied strictly to output logits at the final softmax layer during generation; the model weights are unaffected.
2. Using high temperature without Top-K/P filtering. This can raise the probability of nonsensical or grammatical-error tokens, causing gibberish outputs.

COMMON FOLLOW-UP QUESTIONS

Q: What happens if temperature is set to 0?

It mathematically converges to Greedy Search.

Q: Why is Beam Search expensive for serving?

Because tracking B beams requires replicating the KV Cache state B times, multiplying VRAM footprint.

One-Line Takeaway

Takeaway: Deterministic methods select high-probability paths, while Top-K, Top-P, and Temperature scale logits to inject controlled randomness.

Question 39: Why is Beam Search rarely used in chat LLMs?

[ESSENTIAL]

Conversational Answer

"Beam search is rarely used in chat for two reasons: serving cost and output style. On the serving side, Beam Search requires tracking B candidate sequences (beams) at the same time. This means you have to duplicate the KV Cache for each active beam, which multiplies your VRAM consumption and limits batch capacity. On the quality side, maximizing global probability actually makes chat responses generic and repetitive. Humans don't speak by selecting the most mathematically predictable words; sampling methods like Top-P yield much more natural, engaging conversations."

Intuitive Example

- **Blandness Trap:** For the prompt "How was your day?", Beam Search might select the globally most probable, generic response: "My day was good. I did some work." Top-P sampling allows the model to select slightly lower-probability tokens, yielding a more natural response: "It was pretty busy, actually! I spent most of the afternoon..."

Key Interview Points

- **VRAM Bloat:** Multiplies KV Cache footprint by beam width B .
- **High Latency:** Requires tracking and pruning paths across token steps, breaking parallel batching.
- **Blandness Trap:** Maximizing joint probability favors highly common, generic words, leading to repetitive phrasing.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Beam search tracks B hypotheses. For each step, it expands all B paths into V vocabulary options, sort-selects the top B cumulative log-probability paths, and discards the rest:

$$\text{Score}(y_{1:t}) = \sum_{i=1}^t \log P(y_i \mid y_{1:i-1})$$

This tracking requires active cache synchronization across memory channels.

Production Perspective & Trade-offs

Modern serving frameworks prioritize high concurrency. Replacing Beam Search with sampling techniques (Top-P with temperature) allows serving systems to utilize continuous batching to handle hundreds of concurrent users.

Common Mistakes

1. Assuming Beam Search executes B times slower because of FLOPs. The bottleneck is memory capacity (VRAM allocation for tracking paths), not GPU processing power.
2. Believing Beam Search is bad for all tasks. It is highly effective for code syntax generation or math execution where exact structure matters.

COMMON FOLLOW-UP QUESTIONS

Q: Where is Beam Search still useful?

In highly structured, objective tasks with low-variance targets, such as machine translation, speech-to-text transcription, or code syntax generation.

Q: How does Beam Search impact decoding complexity?

It increases the cost of state management, as the KV Cache must be dynamically copied or pruned when beams are discarded.

One-Line Takeaway

Takeaway: Beam search is avoided in chat because it multiplies KV Cache VRAM consumption and generates generic, robotic responses compared to sampling.

Question 40: What is a context window?

[ESSENTIAL]

Conversational Answer

"The context window is the maximum number of tokens—including both the input prompt and the generated output—that the model can process in a single forward pass. It is bounded by two factors: the mathematical range of your positional encodings (like RoPE frequencies) and the quadratic VRAM footprint of self-attention. If you try to exceed this window, the model will either crash due to Out-of-Memory (OOM) errors or start outputting garbled, repetitive text."

Intuitive Example

- **Shared Window Limits:** If a model has an 8k token context window and you input a 7.5k token document, you only have 500 tokens left for the model's generated response. Exceeding 8k total tokens forces the model to clip early history.

Key Interview Points

- **Attention Limit:** $O(L^2)$ activation memory limit.
 - **Positional Encoding Bounded:** Encodings must resolve coordinate offsets accurately at extreme ranges.
 - **Prefill VRAM Wall:** Ingesting extremely long prompts requires massive initial GPU activation memory.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The memory footprint of storing the raw attention matrix scales quadratically:

$$\text{VRAM}_{\text{Attention}} \propto L^2 \cdot h \cdot N_{\text{layers}}$$

For $L = 128k$ context length, $h = 32$, 32 layers:

$$\text{Matrix Size} = 128000^2 \times 32 \times 32 \times 2 \approx 20.97 \times 10^9 \text{ elements} \approx 42 \text{ GB VRAM}$$

just to store the raw attention weights, creating a physical hardware barrier.

Production Perspective & Trade-offs

To support long contexts (e.g. 1 Million tokens in Gemini), engineering teams use techniques like Grouped-Query Attention (GQA) to shrink KV Cache footprint, FlashAttention to avoid materializing the attention matrix, and sparse or linear attention layers.

Common Mistakes

1. Thinking the context window limits only the input prompt length. The context window is shared; the sum of input prompt tokens and generated output tokens must remain below the limit.
2. Assuming that you can scale context length indefinitely by just adding more GPU VRAM. Positional encodings (like RoPE) must be fine-tuned or scaled (e.g., using Yarn or NTK scaling) to maintain accuracy at long distances.

COMMON FOLLOW-UP QUESTIONS

Q: What is the "needle-in-a-haystack" test?

A benchmark that evaluates whether a model can retrieve a specific fact placed at a random location within a long context window.

Q: How do we extend context length without retraining from scratch?

By using RoPE interpolation techniques (like NTK-aware scaling) to stretch the coordinate space.

One-Line Takeaway

Takeaway: The context window is the maximum shared limit of input and output tokens, bounded by quadratic attention VRAM scaling and positional coordinate ranges.

Question 41: Explain the KV Cache.

[ESSENTIAL]

Conversational Answer

"The KV Cache is an inference optimization that saves us from recomputing past keys and values. During autoregressive decoding, we generate text token-by-token. However, the past tokens in our sequence don't change. Instead of re-calculating the Queries, Keys, and Values for the entire history at every single step, we calculate them once, save the Keys and Values in VRAM, and only calculate the Q, K, V for the single new token. This converts a quadratic $O(L^2)$ generation step into a linear $O(L)$ operation, saving massive compute latency."

Intuitive Example

- **Cache Avoidance:** When generating the 101st token of a response, instead of reprocessing all 100 previous tokens through the network to compute attention, you reuse their cached Keys and Values and only run the projections for the 101st token.

Key Interview Points

- **Compute Savings:** Avoids recalculating keys and values for past prompt/context tokens.
 - **VRAM Trade-off:** Saves compute at the cost of consuming gigabytes of VRAM to store cache state.
 - **Dimensions:** Stores key/value tensors of shape `[B, layers, h_kv, L, d_k]` in GPU global memory.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The VRAM required to store the KV Cache is:

$$\text{VRAM}_{\text{KVCache}} = 2 \times 2 \times B \times L \times n_{\text{layers}} \times n_{\text{heads_kv}} \times d_{\text{head}} \times \text{BytesPerParam}$$

- **VRAM Calculation for Llama-3-70B:**

- Batch Size (B) = 8
- Sequence Length (L) = 4096
- Layers (n_{layers}) = 80
- KV Heads ($n_{\text{heads_kv}}$) = 8 (due to Grouped-Query Attention)
- Head Dim (d_{head}) = 128
- Precision: fp16 (2 Bytes per param)
- Calculation:

$$\text{Total Params} = 2 \times 8 \times 4096 \times 80 \times 8 \times 128 = 5,368,709,120 \text{ params}$$

$$\text{VRAM} = \frac{5,368,709,120 \times 2}{1024^3} \approx 10.0 \text{ GB}$$

- Storing the cache for 8 users at 4k context requires **10.0 GB** of VRAM.

Production Perspective & Trade-offs

Because the KV cache size grows dynamically with each user's generation step, standard static allocation leads to severe VRAM fragmentation and wasted memory. Modern engines use **PagedAttention** to allocate KV Cache memory dynamically in non-contiguous virtual blocks, similar to OS paging.

Common Mistakes

1. Thinking the Query (Q) vectors are cached. Queries are never cached because we only calculate attention for the active query token; we never need past queries.
2. Believing the KV Cache is used during training. During training, teacher forcing exposes all targets in parallel. We process all tokens concurrently using causal masking, so there is no sequential history to cache.

COMMON FOLLOW-UP QUESTIONS

Q: How does batch size affect KV cache VRAM?

It scales linearly; doubling the concurrent users doubles the VRAM allocated for the KV Cache.

Q: What happens when the GPU runs out of VRAM for the KV Cache?

The serving framework must either swap cache blocks to CPU system memory (adding latency) or pre-empt requests.

One-Line Takeaway

Takeaway: The KV Cache stores Key and Value activations for past tokens in VRAM to convert sequential decoding compute from $O(L^2)$ to $O(L)$.

Question 42: Why is the first generated token slower than subsequent tokens? (Prefill vs Decode)

[ESSENTIAL]

Conversational Answer

"This is due to the difference between the Prefill and Decode phases. The first token is generated during the **Prefill** phase, where the model processes the entire input prompt in parallel to compute the initial KV Cache. This phase is compute-bound, saturating the GPU Tensor Cores with massive matrix multiplications. Subsequent tokens are generated during the **Decode** phase, which processes only one token at a time. Decode is memory-bandwidth bound: the GPU spends most of its time waiting to load model weights from memory to process that single token, leading to higher latency per token."

Intuitive Example

- **Prefill vs Decode:**
 - **Prefill (First Token):** Reading a 1,000-word essay to write the first word of summary. (High parallel compute).
 - **Decode (Subsequent Tokens):** Writing the rest of the summary word-by-word. (Sequential weight loading).

Key Interview Points

- **Prefill (TTFT):** Parallel processing of prompt; compute-bound, drives Time-to-First-Token.

- **Decode (ITL):** Sequential single-token generation; memory-bandwidth bound, drives Inter-Token Latency.
 - **Roofline Boundary:** Prefill saturates Tensor Cores; Decode sits idle waiting for memory transfer.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Roofline Analysis:**

- Operational Intensity is defined as:

$$\text{Intensity} = \frac{\text{FLOPs}}{\text{Bytes accessed}}$$

- Prefill processes the prompt in parallel, yielding high operational intensity (> 150 FLOP/Byte) which saturates GPU compute cores.
- Decode processes a single query vector against the entire model weights, yielding an intensity of ≈ 1.0 FLOP/Byte, which sits far below the GPU compute ceiling, making it memory-bandwidth bound.

Production Perspective & Trade-offs

Because prefill and decode have different bottlenecks, modern serving frameworks use **Chunked Prefill** to split large prompts into smaller blocks, mixing them with decode requests to maintain balanced GPU utilization.

Common Mistakes

1. Assuming decode is slow because it performs more FLOPs. Decode performs significantly fewer FLOPs than prefill, but runs slower because it cannot parallelize weight retrieval.
2. Thinking that prompt processing is sequential. The entire prompt is computed in parallel in a single forward pass.

COMMON FOLLOW-UP QUESTIONS

Q: How does batch size affect decode efficiency?

A larger batch size increases operational intensity during decode (processing multiple query vectors against the same weights), improving GPU utilization.

Q: What is TTFT and why is it critical?

Time-to-First-Token; it measures user-perceived responsiveness and is determined by prompt length and prefill speed.

One-Line Takeaway

Takeaway: The first token is slower because the prefill phase processes the entire prompt in parallel (compute-bound), while decoding subsequent tokens is memory-bandwidth bound.

Question 43: How do Multi-Query Attention (MQA) and Grouped-Query Attention (GQA) reduce inference cost?

[ESSENTIAL]

Conversational Answer

"MQA and GQA reduce inference cost by shrinking the memory size of the KV Cache. In Multi-Head Attention, every Query head has its own Key and Value head. Multi-Query Attention (MQA) goes to the extreme: it uses a single Key and Value head shared across *all* Query heads, which shrinks the cache size by up to 98% but degrades quality. Grouped-Query Attention (GQA) is the modern sweet spot: it groups Query heads into clusters and assigns a single Key/Value head per group. GQA recovers almost all model quality while delivering massive VRAM savings and higher serving throughput."

Intuitive Example

- **Attention Head Mapping:**
 - **MHA:** 32 Query heads map to 32 Key/Value heads. (Standard large KV Cache).
 - **MQA:** 32 Query heads share 1 Key/Value head. (Tiny KV Cache, minor quality drop).
 - **GQA** (8 groups): 32 Query heads grouped in clusters of 4, each cluster sharing 1 Key/Value head (total of 8 KV heads). (Optimal balance).

Key Interview Points

- **MHA:** H Query heads, H Key heads, H Value heads.
- **MQA:** H Query heads, 1 Key head, 1 Value head. Shrinks cache size by H .
- **GQA:** H Query heads, G Key/Value heads ($G = H/\text{Group_Size}$).
- **Inference Speed:** Smaller KV caches reduce memory bandwidth demand, speeding up decoding.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Comparison Table:**

Attention Type	Query Heads	Key/Value Heads	KV Cache Size Ratio	Model Quality
Multi-Head (MHA)	H	H	1.0 (Baseline)	100% (Highest)
Grouped-Query (GQA)	H	$G = H/\text{Group}$	G/H (e.g. 25%)	Near-equivalent
Multi-Query (MQA)	H	1	$1/H$ (e.g. 3%)	Degraded

The KV Cache shapes in memory are:

$\text{MHA Shape} = [B, L, H, d_k], \quad \text{GQA Shape} = [B, L, G, d_k], \quad \text{MQA Shape} = [B, L, 1, d_k]$

Production Perspective & Trade-offs

Reducing the KV cache size directly lowers the memory-bandwidth bottleneck during decoding, allowing serving engines (like vLLM) to host larger batch sizes (more concurrent users) in the same GPU VRAM, lowering infrastructure costs.

Common Mistakes

- 1. Thinking GQA/MQA reduces model parameters significantly. They only reduce the projection layer parameters for Keys and Values, which represents less than 2% of total model parameters. The primary savings are in KV Cache activation memory during inference.
- 2. Believing GQA can be applied to any pre-trained model at inference time. The model must be pre-trained with grouped projections, or converted via specialized fine-tuning.

COMMON FOLLOW-UP QUESTIONS

Q: Why does GQA perform better than MQA?

Because having multiple KV heads (e.g. 8) allows the model to retain distinct contextual subspaces, whereas a single KV head forces all query heads to share a single channel.

Q: What is the KV cache reduction factor for a model with 32 query heads and 8 KV heads?

It reduces the KV Cache size by a factor of 4 ($32/8$).

One-Line Takeaway

Takeaway: GQA groups query heads to share Key/Value projections, shrinking the KV Cache VRAM footprint to enable larger batch sizes and higher decoding throughput.

8. Modern LLM Architectures & Mixture of Experts (Q44–Q47)

Question 44: Compare GPT, BERT, T5, and Llama.

[ESSENTIAL]

Conversational Answer

"I'd compare them based on their block structures and attention mechanics. BERT is an encoder-only model with bidirectional attention, ideal for language understanding and embeddings. GPT is a decoder-only model with causal masking, optimized for autoregressive text generation. T5 is an encoder-decoder model that treats all tasks as text-to-text transformation, which is highly effective for translation. Llama is the modern standard decoder-only model, improving on GPT by using pre-activation RMSNorm, SwiGLU, RoPE, and GQA for highly stable training and fast inference."

Intuitive Example

- **Architecture Differences:**
 - **BERT:** In "*Paris is the [MASK] of France*", looks both ways to predict "*capital*".
 - **GPT:** In "*Paris is the*", only looks left to predict "*capital*".
 - **T5:** Reads French "*Paris est la capitale*", cross-attends, and generates English "*Paris is the capital*".
 - **Llama:** Processes "*Paris is the*" causally, but rotates Query/Key slices for relative distance modeling.

Key Interview Points

- **BERT:** Encoder-only, bidirectional context, MLM pre-training, understanding focus.
 - **GPT:** Decoder-only, causal masking, autoregressive next-token prediction, generation focus.
 - **T5:** Encoder-decoder, text-to-text framework, cross-attention layers.
 - **Llama:** Modernized decoder-only, incorporating RMSNorm, RoPE, SwiGLU, and GQA.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Topology Comparison Table:**

Feature	BERT	GPT (original)	T5	Llama 3
Type	Encoder-only	Decoder-only	Encoder-Decoder	Decoder-only
Attention	Bidirectional	Causal Masked	Bidirectional + Causal	Causal Masked (GQA)
Norm Layer	Post-LN (LayerNorm)	Post-LN (LayerNorm)	Pre-LN (LayerNorm)	Pre-LN (RMSNorm)
Position	Absolute Learned	Absolute Learned	Relative Buckets	Relative (RoPE)
FFN Activ.	GELU	GELU	ReLU / GEGLU	SwiGLU

Production Perspective & Trade-offs

Decoder-only models dominate the market because they share parameters across prompt processing and generation, making them flexible few-shot learners. However, they suffer from sequential inference bottlenecks. Encoder-only models are significantly faster and cheaper to serve for simple extraction or classification tasks.

Common Mistakes

1. Thinking Llama uses an encoder. Llama is a decoder-only model; it has no cross-attention or encoder blocks.
2. Assuming T5 is slower than GPT because of the two-part structure. For structured translation, T5 is highly parameter-efficient.

COMMON FOLLOW-UP QUESTIONS

Q: Why did decoder-only architectures win the general assistant race over encoder-decoders?

Because a single, homogeneous block structure is easier to scale on GPUs and naturally supports zero-shot instructions.

Q: What is the main serving difference between BERT and Llama?

BERT runs in a single parallel step (constant latency), whereas Llama runs sequential decoding loops (latency scales with output tokens).

One-Line Takeaway

Takeaway: BERT is bidirectional for understanding; GPT/Llama are causal decoders for generation; T5 combines both for sequence-to-sequence translation.

Question 45: What innovations did Llama introduce?

[ESSENTIAL]

Conversational Answer

"Llama didn't change the core Transformer concept, but it modernized the architecture by importing four key innovations that stabilize training and speed up serving. First, it moved normalization to the input of each block using RMSNorm, which is faster because it skips the mean calculation. Second, it replaced GELU with SwiGLU, which uses a gated multiplication to increase representation capacity. Third, it introduced RoPE for relative positional modeling. Finally, it used Grouped-Query Attention to drastically shrink the KV Cache memory footprint during serving."

Intuitive Example

- **Modernizing GPT:** Llama-3 takes the classic GPT-2 structure and replaces LayerNorm with RMSNorm, absolute embeddings with RoPE, and standard attention with GQA. This allows it to scale context length to 128k stably on standard GPU clusters.

Key Interview Points

- **RMSNorm:** Faster normalization by skipping mean-centering reductions.
 - **SwiGLU:** Gated non-linear projections for higher capacity.
 - **RoPE:** Relative positional rotations with excellent context length extrapolation.
 - **GQA:** Shrunk KV Cache memory size, enabling high serving concurrency.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Llama scales FFN parameter counts while maintaining compute density by setting the intermediate dimension to:

$$d_{\text{ffn}} = \left\lfloor \frac{2}{3} \cdot 4d_{\text{model}} \right\rfloor = \left\lfloor \frac{8}{3} d_{\text{model}} \right\rfloor$$

This matches the parameter count of standard FFNs while implementing SwiGLU gating.

Production Perspective & Trade-offs

GQA combined with RMSNorm allows Llama-3-8B to deliver nearly 2x the inference token-generation throughput of Llama-1-7B on the same GPU hardware.

Common Mistakes

1. Thinking Llama uses standard LayerNorm. Llama uses RMSNorm exclusively.
2. Assuming Llama models are trained bidirectionally. They are causally masked next-token predictors.

COMMON FOLLOW-UP QUESTIONS

Q: What is the difference between Llama-1, Llama-2, and Llama-3?

Llama-1 introduced base innovations; Llama-2 doubled context to 4k and added MQA/GQA on larger sizes; Llama-3 upgraded vocabulary size to 128k, added GQA to all sizes, and extended context to 8k+.

Q: Why does Llama use SwiGLU instead of standard FFN?

Because gating projections improve training efficiency and representation capacity.

One-Line Takeaway

Takeaway: Llama optimizes decoder-only performance using Pre-activation RMSNorm, SwiGLU, Rotary Positional Embeddings, and Grouped-Query Attention.

Question 46: What is Mixture of Experts (MoE)? Explain Dense vs Sparse models.

[ESSENTIAL]

Conversational Answer

"A dense model activates every single parameter for every token. A sparse Mixture of Experts, or MoE, model scales capacity without scaling compute cost. It replaces the standard FFN layer in the Transformer block with multiple parallel 'experts'—which are just independent FFNs. For every token, a routing network calculates logits and selects only a small subset of experts, like the top-2, to process that token. This allows you to scale a model to 100 Billion parameters, while the compute cost per token remains equivalent to a much smaller 20 Billion parameter model."

Intuitive Example

- **Dynamic Expert Routing:** In processing the phrase "*compute GPU FLOPs*", the router redirects the token "*GPU*" to the hardware and mathematics experts, while routing "*French*" in another sentence to the linguistics expert.

Key Interview Points

- **Dense Models:** All parameters activated per token (100% parameter utilization).

- **Sparse MoE:** Router dynamically routes tokens to K active experts out of N total.
- **Active vs. Total Parameters:** A model can have 141B total parameters but only activate 37B parameters per token (e.g., Mixtral 8x22B).

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

- **Comparison Table:**

Metric	Dense Model	Sparse MoE Model
Parameter Utilization	100% on every token	Sparse ($Top - K$ active experts)
VRAM Footprint	Scales with active parameters	Scales with total parameters
Tokens/sec Speed	Fixed by model parameter size	Matches active parameter size (faster)
Training Complexity	Standard backpropagation	Complex expert routing & balancing

For each token x , routing weights are computed using softmax over the top- K experts:

$$G(x) = \text{softmax}(\text{KeepTopK}(x \cdot W_g, K))$$

The output is the weighted sum:

$$y = \sum_{i \in \text{selected}} G(x)_i E_i(x)$$

Production Perspective & Trade-offs

MoE models require loading all experts into VRAM. A 141B parameter MoE model requires multiple GPUs just to hold the weights in memory, making hosting expensive for low-concurrency workloads.

Common Mistakes

1. Believing MoE is an ensemble model where outputs of all experts are averaged. MoE is sparse; only the top-K selected experts are executed for any single token.
2. Thinking MoE reduces VRAM requirements. It reduces compute (FLOPs) per token, but increases VRAM requirements because all experts must remain in GPU memory.

COMMON FOLLOW-UP QUESTIONS

Q: How does routing affect batching?

Since different tokens are routed to different experts, batch sizes per expert fluctuate dynamically. Serving engines must use expert-level grouping and padding to maintain efficiency.

Q: What is Top-1 vs. Top-2 routing?

Top-1 routes to a single expert (fastest); Top-2 routes to two experts and mixes their outputs (higher quality).

One-Line Takeaway

Takeaway: MoE replaces FFN layers with parallel experts, routing each token to a sparse subset to scale parameter capacity without increasing compute cost.

Question 47: What is Router Collapse in MoE models, and how do we mitigate it?

[ESSENTIAL]

Conversational Answer

"Router collapse is a common training failure where the routing network gets stuck over-selecting a few popular experts early on. As these experts get chosen more, they receive more gradient updates and improve faster. This makes the router favor them even more in a self-reinforcing loop, leaving the other experts untrained and unused. We mitigate this during training by adding a **Load Balancing Loss** to the objective function, which penalizes the router for non-uniform expert selection, forcing a balanced token distribution."

Intuitive Example

- **Popularity Trap:** If the router sends 99% of tokens to Expert 1 because it was slightly better initialized, Expert 1 does all the learning. The other experts remain random weights. The model effectively collapses back into a standard dense model with a single active FFN.

Key Interview Points

- **Unbalanced Utilization:** A few experts process 99% of the tokens, while others sit idle.
 - **Self-Reinforcing Loop:** Popular experts get updated more, causing the router to favor them.
 - **Mitigation:** Load Balancing (Auxiliary) Loss and capacity clamping.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The load balancing loss evaluates the dot product of the fraction of tokens routed to expert i (f_i) and the routing probability assigned to expert i (P_i) across a batch:

$$\mathcal{L}_{\text{aux}} = N \sum_{i=1}^N f_i \cdot P_i$$

Minimizing this product forces the token distribution f and probabilities P toward a uniform distribution ($1/N$).

Production Perspective & Trade-offs

During inference, serving engines enforce an "Expert Capacity" parameter. If an expert receives more tokens than its capacity threshold, the excess tokens are dropped or routed to second-choice experts to prevent memory overflow.

Common Mistakes

1. Thinking router collapse is resolved at inference time. Mitigations must be enforced during pre-training using auxiliary losses; post-hoc adjustments cannot fix untrained experts.
2. Believing load balancing loss degrades model performance. It stabilizes training by utilizing the full representation capacity of all experts.

COMMON FOLLOW-UP QUESTIONS

Q: What is Expert Capacity?

The maximum number of tokens an expert is allowed to process in a single batch during serving to prevent load imbalance on GPUs.

Q: Can we use static routing?

Yes, some architectures use hash-based routing that bypasses learned routers entirely, though this reduces semantic grouping.

One-Line Takeaway

Takeaway: Router collapse occurs when the routing network over-selects a few experts; we mitigate it during training using a Load Balancing Auxiliary Loss.

9. Reasoning & Deep Thinking Models (Q48–Q50)

Question 48: How do reasoning models (e.g. DeepSeek-R1, OpenAI o1) scale Test-Time Compute (TTC)?

[ESSENTIAL]

Conversational Answer

"Reasoning models scale Test-Time Compute by shifting compute from training to inference. Instead of generating the final answer immediately, the model is trained via reinforcement learning to generate a long, structured Chain-of-Thought (CoT) trace—represented as hidden thinking tokens—before outputting the final response. This allows us to scale compute dynamically at run-time: for a hard math problem, we can let the model generate hundreds of thinking tokens to search, verify, and correct its reasoning path before committed output."

Intuitive Example

- **TTC Scaling:** When asked to solve a complex coding bug, instead of predicting code in 1 step, the model generates 800 `<think>` tokens: *"Let's test edge case A... wait, that fails. Let me rewrite function X. Okay, now it passes."* Only then does it output the correct code.

Key Interview Points

- **Inference Scaling:** Shifting FLOP expenditures from offline pre-training to online token generation.
 - **Chain of Thought (CoT):** Generating hidden or explicit `<think>` tokens representing intermediate steps.
 - **Search and Rollouts:** Using RL-guided path expansion to explore and verify options.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Traditional scaling laws focus on training compute $C = 6ND$. Reasoning models add a test-time scaling law, where accuracy scales with thinking sequence length L_{thinking} :

$$\text{Accuracy} \propto f(L_{\text{thinking}})$$

This allows a smaller base model (e.g. 32B parameters) to outperform dense 70B models by spending 10x more tokens on reasoning during inference.

Production Perspective & Trade-offs

Generating hundreds of thinking tokens per query drastically increases serving costs. The KV cache size grows linearly, and sequence execution times extend, increasing latency and reducing serving concurrency.

Common Mistakes

1. Believing reasoning models use separate external solvers to write code. The reasoning, self-correction, and math execution occur entirely within the LLM's own autoregressive token generation path.
2. Thinking that more thinking tokens always improve output quality. For simple queries (e.g., "*What is capital of France?*"), generating thinking tokens adds latency and cost without improving accuracy.

COMMON FOLLOW-UP QUESTIONS

Q: What is GRPO and why is it used?

Group Relative Policy Optimization, an RL algorithm that samples multiple outputs per prompt and calculates rewards relative to the group average, eliminating the memory overhead of a separate Critic network.

Q: How are thinking tokens hidden from users?

In production APIs, the tokens within ``...`` blocks are stripped from the final user response payload.

One-Line Takeaway

Takeaway: Reasoning models scale Test-Time Compute by generating sequential thinking tokens to explore, verify, and correct logical paths during inference.

Question 49: How do reasoning models perform self-correction and backtracking during generation?

[ESSENTIAL]

Conversational Answer

"Self-correction and backtracking are not hardcoded; they are learned behaviors reinforced by Reinforcement Learning. During RL training, the model is rewarded when it catches its own mistake and successfully corrects it to reach the correct answer. The model learns to output phrases like 'Wait, this calculation is wrong' and backtrack. Because the previous error tokens are in its context history, the attention layers can process the mistake and adjust subsequent token probabilities to head down a different logical path."

Intuitive Example

- **Backtracking:** Inside the thinking trace, the model generates: *"The prime factor of 91 is 13... wait, is 7 also a factor? $91/7 = 13$, yes, so the factors are 7 and 13."* The model caught its initial omission and corrected it before printing the final answer.

Key Interview Points

- **Explicit Traces:** Self-correction is expressed directly as text tokens within the generation path.
 - **RL Incentives:** Accuracy rewards reinforce the behavior of catching errors and correcting them.
 - **Infinite Loop Risks:** The model can get stuck in repetitive error-correction loops, requiring context window limits or penalties.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The model's weights remain frozen during inference. Self-correction is driven entirely by processing previous text tokens in the attention context window. The policy learns to evaluate intermediate steps. If the reward signal at training checks output accuracy $\mathcal{A}$, the model gets high reward only when it reaches the correct final answer. If it hits an error and outputs "Let me re-try..." to reach the correct answer, it receives the same reward as a direct path, encouraging self-correction.

Production Perspective & Trade-offs

Backtracking increases sequence length. If a model spends 800 tokens self-correcting, the user must wait for the decode sequence to finish, requiring streaming UI layouts to display the active thinking process.

Common Mistakes

1. Thinking self-correction requires modifying active attention weights at runtime. The attention weights are frozen; self-correction is driven entirely by context window history processing of previously generated error tokens.
2. Assuming self-correction is 100% reliable. In practice, models can still hallucinate that they fixed an error when they actually introduced a new one.

COMMON FOLLOW-UP QUESTIONS

Q: How do we prevent models from cheating by outputting long, meaningless thinking traces to pad compute?

By adding a penalty reward proportional to the length of the thinking sequence (length penalty) to force concise reasoning.

Q: Can we force backtracking using search trees?

Yes, by running Monte Carlo Tree Search (MCTS) to generate multiple thinking branches and selecting the highest-rated branch.

One-Line Takeaway

Takeaway: Self-correction is an emergent behavior learned via RL, where the model uses its generated error history in the context window to redirect attention.

Question 50: Contrast Process-Supervised Reward Models (PRMs) and Outcome-Supervised Reward Models (ORMs).

[ESSENTIAL]

Conversational Answer

"Outcome-Supervised Reward Models, or ORM, only grade the final answer—they look at the end result and give a binary thumbs-up or down. Process-Supervised Reward Models, or PRMs, grade every individual step in the reasoning chain. While ORMs are easy to automate (like running code compilers), they can reward correct answers reached through bad logic. PRMs directly reward correct step-by-step thinking, which reduces hallucinations and helps RL algorithms optimize complex search trees."

Intuitive Example

- **Step-by-Step Grading:**
 - **ORM:** Math answer is 12 → Reward = 1.0 (Even if the model added $5 + 5$ to get 12 by mistake).
 - **PRM:** Step 1: $2 \times 3 = 6$ (Reward 1.0). Step 2: $6 \times 2 = 12$ (Reward 1.0). This ensures the logic is sound at every node.

Key Interview Points

- **ORM:** Outcome focus. Simple, prone to rewarding false positives (lucky guesses).

- **PRM:** Process focus. Step-by-step reinforcement, reduces hallucinations, requires high-quality step-level labeling.
 - **Reasoning Alignment:** PRMs are essential for guiding models to generate correct, logical thinking traces.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Let a reasoning trace contain steps $S = [s_1, s_2, \dots, s_k, \text{Final Answer}]$.

- **ORM Reward:**

$$\mathcal{R}_{\text{ORM}} = r(\text{Final Answer} \mid S)$$

- **PRM Reward:**

$$\mathcal{R}_{\text{PRM}} = \prod_{i=1}^k r(s_i \mid s_{<i})$$

where each step s_i receives an independent probability score of being mathematically or logically correct.

Production Perspective & Trade-offs

Training a PRM requires massive human-in-the-loop or LLM-as-a-judge labeling to score millions of individual reasoning steps, making it significantly more expensive to train than simple ORMs (which use automated code compilers or unit tests).

Common Mistakes

1. Believing PRMs are executed during inference. PRMs are used during training (RL reward signals) or during verification search (ranking candidate paths), but are not part of the base model's forward generation pass.
2. Assuming ORMs are useless. ORMs are highly effective for code syntax or math compilation where final correctness is easily verifiable.

COMMON FOLLOW-UP QUESTIONS

Q: How do we automate step-level labeling for PRMs?

By using a stronger model (like GPT-4) as a judge to evaluate and score the validity of each step in the generated trace.

Q: What is Active Reward Modeling?

Dynamically updating the reward model during RL training to address gaps where the generator learns to exploit the reward function.

One-Line Takeaway

Takeaway: ORMs reward the final output, while PRMs reward every individual reasoning step to prevent correct answers from being reached via faulty logic.

10. Limitations, Evaluation, & Production (Q51–Q53)

Question 51: Why do LLMs hallucinate, and how can hallucinations be reduced?

[ESSENTIAL]

Conversational Answer

"LLMs hallucinate because they are next-token probability predictors trained on noisy web text. They prioritize fluency and high probability over factual truth. Since they lack an internal database of truth and process prompts causally, they default to plausible-sounding completions when context is missing. We reduce hallucinations using **Retrieval-Augmented Generation (RAG)** to ground prompts in external documents, using RL preference alignment (DPO/RLHF) to teach the model to say 'I don't know,' and using decoding limits like low temperature."

Intuitive Example

- **Grounding Context:**
 - **Without RAG:** "Who won the local golf tournament in my city yesterday?" → Model guesses a name based on past patterns (hallucination).
 - **With RAG:** Vector search retrieves yesterday's news article and appends it to the prompt. The model reads the text and extracts the correct name.

Key Interview Points

- **Probabilistic Nature:** LLMs optimize for fluency and token probability, not factual truth.
 - **Exposure Bias:** Model deviations from truth compound sequentially during decoding.
 - **Mitigation:** RAG (grounding), RL alignment (honesty optimization), and system prompts.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Facts are compressed into static parameter weights during training. When retrieving a fact, the model computes:

$$P(\text{Fact} \mid \text{Prompt}) \propto \text{Activation weights}$$

If the training corpus had low density or conflicting information for the fact, the softmax probability will be spread across multiple candidates, causing the model to generate a blend of facts (hallucination). RAG resolves this by injecting the literal text into the input context, converting a parameter recall task into a reading comprehension task.

Production Perspective & Trade-offs

RAG requires a retrieval database pipeline (vector index lookup), which adds latency to the system. The trade-off is higher accuracy and groundability in exchange for extra infrastructure cost and slower Time-to-First-Token (TTFT).

Common Mistakes

1. Thinking fine-tuning on fact databases completely resolves hallucinations. Fine-tuning can actually increase hallucinations (hallucination by injection) if the model is forced to memorize facts it lacks the parameter capacity to store.
2. Assuming low temperature completely prevents hallucinations. It only makes the model output the most probable tokens, which can still be factual errors if the model weights contain bad facts.

COMMON FOLLOW-UP QUESTIONS

Q: What is "hallucination snowballing"?

When a model makes a small factual error, this error token is appended to the input history, forcing all subsequent tokens to align with the error to maintain logical coherence, compounding the hallucination.

Q: How does RLHF help reduce hallucinations?

By training the reward model to heavily penalize confident incorrect statements and reward responses like **"I do not know"**.

One-Line Takeaway

Takeaway: Hallucinations occur because LLMs optimize for token probability over factual truth; we mitigate them using RAG grounding and honesty-oriented RL alignment.

Question 52: How are LLMs evaluated?

[ESSENTIAL]

Conversational Answer

"We evaluate LLMs using a mix of metrics. For raw language prediction, we use **Perplexity** to measure how confident the model is on a test set. For general knowledge, we use standardized benchmarks like **MMLU** (multiple-choice), **GPQA** (difficult graduate-level science), and **AIME** (math olympiad). In production, we rely on **LLM-as-a-Judge** where we use a larger model like GPT-4 to grade responses, and **LMSYS Chatbot Arena**, which collects real-time human votes to calculate Elo ratings."

Intuitive Example

- **Elo Rating:** In the Chatbot Arena, a user inputs a prompt, and two anonymous models generate answers. The user votes on the better answer. If Model A consistently beats Model B , its Elo rating rises, providing a highly reliable measure of human preference.

Key Interview Points

- **Perplexity:** Log probability of test text; lower is better.
- **GPQA/AIME:** High-difficulty reasoning benchmarks designed to prevent contamination.
- **LLM-as-a-Judge:** Scalable evaluation method; can be biased toward longer or self-generated style outputs.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Perplexity is the exponential of the cross-entropy loss $H(X)$ over a sequence of length N :

$$\text{PPL}(X) = e^{H(X)} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \ln P(x_i | x_{<i}) \right)$$

If the model is perfectly certain, $\text{PPL} = 1.0$.

Production Perspective & Trade-offs

Modern LLMs are trained on vast web scrapes, which often contain test sets from popular benchmarks (MMLU). GPQA is crucial because it keeps its questions hidden from public web indexing to guarantee clean evaluation.

Common Mistakes

1. Relying on BLEU/ROUGE for conversational chat models. These metrics require exact word match overlaps, failing to capture semantic synonyms or conversational quality.
2. Assuming a high MMLU score guarantees a model is good in production. Benchmark contamination and overfitting can inflate scores without improving real-world capability.

COMMON FOLLOW-UP QUESTIONS

Q: What are the biases of LLM-as-a-Judge?

Position bias (prefers the first option), verbosity bias (prefers longer answers), and self-preference bias (prefers its own generated style).

Q: How do we prevent benchmark contamination?

By hashing test questions and checking if they appear in the pre-training text corpus.

One-Line Takeaway

Takeaway: LLMs are evaluated using perplexity for prediction quality, reasoning benchmarks like GPQA to avoid contamination, and human Elo ratings for preference alignment.

Question 53: Walk through the complete LLM inference pipeline and explain optimization techniques for lower latency.

[ESSENTIAL]

Conversational Answer

"The inference pipeline has four main stages. First, the user prompt is tokenized on the CPU and sent to the GPU. Second, we run the **Prefill** phase: the model processes all prompt tokens in parallel to generate the first token and save the initial Key-Value Cache. Third, we run the **Decode** loop: we sequentially generate tokens one-by-one, reusing the KV Cache at each step. Finally, the output tokens are converted back to text. To optimize latency, we use **Quantization** to shrink weights, **GQA** to reduce the KV Cache footprint, **FlashAttention** to optimize GPU memory access, and **PagedAttention** to enable batching without memory waste."

Intuitive Example

- **Inference Loop:**

- Input prompt: "Translate: Hello" → Tokenized to [1050, 4531] .
- Prefill: Model processes [1050, 4531] in parallel, outputs token [8432] ("Bonjour"), caches Key-Values.
- Decode: Input [8432] , reuse cached KV, output [2] (<eos>).
- Detokenize: Output "Bonjour" .

Key Interview Points

- **Inference Pipeline:** Tokenization → Prefill (parallel) → Decode (sequential) → De-tokenization.
 - **Quantization:** Converts weights from FP16 to FP8 or INT4, reducing VRAM footprint and memory bandwidth limits.
 - **PagedAttention:** Manages KV cache dynamically in non-contiguous virtual memory blocks.
 - **Speculative Decoding:** Uses a small draft model to generate candidate tokens, validating them in parallel to bypass sequential latency walls.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Decode is memory-bandwidth bound, meaning generation latency scales with model size and memory speed:

$$\text{Latency}_{\text{Decode}} \propto \frac{\text{Model Size (Bytes)}}{\text{GPU Memory Bandwidth (Bytes/sec)}}$$

Quantizing a 70B model from 16-bit to 4-bit shrinks model size from 140 GB to 35 GB, yielding an immediate ~4x speedup in token generation latency.

Production Perspective & Trade-offs

Traditional batching groups requests statically, forcing users to wait for the longest generation to complete. Continuous batching schedules requests at the iteration level, inserting new prompts and ejecting finished generations dynamically at every step, boosting GPU throughput by up to 4x.

Common Mistakes

1. Thinking speculative decoding reduces the total FLOPs computed. Speculative decoding actually computes *more* FLOPs (due to double forward passes and validation checks), but runs faster because it reduces the number of sequential memory-bandwidth bound steps.

2. Assuming CPU-GPU transfer happens at every token step. Weights are loaded once to GPU memory; only input/output tokens transfer during generation, which adds negligible latency.

COMMON FOLLOW-UP QUESTIONS

Q: What is Tensor Parallelism?

Splitting the model's weight matrices horizontally or vertically across multiple GPUs (within a single node) to run parallel matrix multiplications, reducing single-GPU VRAM demand.

Q: How does vLLM optimize serving?

By using PagedAttention to eliminate KV cache fragmentation, allowing up to 10x higher batch sizes in the same GPU VRAM.

One-Line Takeaway

Takeaway: Inference combines parallel prefill and sequential decode; we optimize it using quantization, GQA, FlashAttention, and PagedAttention to maximize memory throughput.

LLM Foundations: Final 2-Page Revision Sheet

1. Quick-Recall One-Line Takeaways

Q#	Topic Area	One-Line Takeaway
Q1	Evolution of NLP	Transformers replaced RNNs because self-attention eliminates the sequential training bottleneck, allowing efficient GPU parallelization.
Q2	Evolution of NLP	RNN Seq2Seq models are limited by sequential training constraints and an information bottleneck that compresses long contexts into a single fixed vector.
Q3	Evolution of NLP	Word2Vec and GloVe generate static word-level embeddings, while FastText operates on character n-grams to handle OOV words.
Q4	Evolution of NLP	Attention was introduced to resolve the fixed-vector information bottleneck by allowing the decoder to dynamically query all encoder states.
Q5	Transformer Core	The Transformer is a recurrence-free architecture relying on self-attention, FFNs, and residual connections to process sequence context in parallel.

Q#	Topic Area	One-Line Takeaway
Q6	Transformer Core	Encoder-only models are bidirectional for comprehension; decoder-only models are causal for generation; encoder-decoder models link both.
Q7	Transformer Core	GPT is decoder-only because a single, causally masked stack of layers is highly homogeneous and optimal for scaling next-token prediction.
Q8	Transformer Core	FFNs process each token position independently using non-linear activations to project features, acting as the primary parameter store.
Q9	Transformer Core	Residual connections create additive shortcut pathways that allow gradients to backpropagate directly through deep stacks without vanishing.
Q10	Transformer Core	RMSNorm replaces LayerNorm by normalizing inputs using only their root mean square, skipping mean calculation to save GPU memory bandwidth.
Q11	Transformer Core	Pre-LN normalizes activations prior to block execution, keeping the residual path clean to stabilize gradient flow in deep networks.
Q12	Transformer Core	Modern LLMs use SwiGLU because its smooth, gated projection avoids dead neurons and increases representational capacity.
Q13	Attention	Self-attention dynamically routes context-aware representations across all tokens in a sequence using query-key compatibility matching.
Q14	Attention	Query acts as a semantic search term, Key as the database index, and Value as the content retrieved based on the query-key match.
Q15	Attention	Scaling attention scores by the square root of the head dimension prevents softmax saturation and vanishing gradients during training.
Q16	Attention	Causal masking blocks information flow from future tokens by adding negative infinity to future scores before the softmax operation.
Q17	Attention	Self-attention is quadratic because calculating pairwise similarity scores across all positions creates an $L \times L$ matrix bottleneck.
Q18	Attention	Multi-Head Attention splits the hidden dimension into parallel heads to capture diverse syntactic and semantic relationships simultaneously.
Q19	Attention	Attention heads specialize naturally in local, syntactic, and semantic relationships, as well as pattern-copying induction mechanisms.

Q#	Topic Area	One-Line Takeaway
Q20	Attention	Self-attention routes information within a single sequence, while cross-attention links the decoder to the encoder's static representations.
Q21	Attention	FlashAttention accelerates attention by loading tokens block-by-block into local SRAM, computing online softmax without HBM writes.
Q22	Attention	FlashAttention recomputes the forward attention matrix in fast SRAM during the backward pass, trading cheap FLOPs to avoid slow HBM reads.
Q23	Positional Encoding	Transformers require positional encodings because self-attention is permutation invariant and cannot distinguish token order without coordinates.
Q24	Positional Encoding	RoPE rotates Query and Key vector slices in 2D to mathematically build relative distance dependencies directly into attention dot products.
Q25	Positional Encoding	Learned embeddings do not extrapolate; RoPE rotates vector slices for relative representation; ALiBi adds a linear distance penalty.
Q26	Tokenization	BPE is an iterative statistical algorithm that merges the most frequent adjacent character/subword pairs to build a compressed vocabulary.
Q27	Tokenization	BPE uses frequency merges; WordPiece maximizes likelihood; SentencePiece processes raw byte streams, preserving whitespace.
Q28	Tokenization	Token count dictates computational complexity and VRAM consumption due to quadratic self-attention and linear KV cache memory scaling.
Q29	Embeddings	Contextual embeddings generate dynamic, neighborhood-aware representations for tokens at runtime by passing them through self-attention layers.
Q30	Embeddings	Cosine similarity measures directional alignment in embedding space while ignoring vector magnitudes, isolating semantic content.
Q31	Training & Tuning	GPT is trained by projecting hidden states to vocabulary logits and using causal masking to compute next-token cross-entropy loss.
Q32	Training & Tuning	Teacher Forcing feeds ground-truth tokens as inputs at every training step to stabilize gradient updates and enable parallel sequence calculations.

Q#	Topic Area	One-Line Takeaway
Q33	Training & Tuning	Pre-training builds general language representations; instruction tuning aligns models to follow prompts; fine-tuning adapts them to tasks.
Q34	Training & Tuning	SFT adapts base models to chat formats by evaluating next-token cross-entropy loss exclusively on target response tokens.
Q35	Training & Tuning	DPO simplifies RLHF alignment by mathematically reformulating preference optimization into a direct binary cross-entropy loss.
Q36	Training & Tuning	Chinchilla laws show parameters and data should scale equally, but production serving costs favor over-training smaller models.
Q37	Inference	Autoregressive generation generates text sequentially by repeatedly appending the predicted next token to the input.
Q38	Inference	Deterministic methods select high-probability paths, while Top-K, Top-P, and Temperature scale logits to inject controlled randomness.
Q39	Inference	Beam search is avoided in chat because it multiplies KV Cache VRAM consumption and generates generic, robotic responses.
Q40	Inference	The context window is the maximum shared limit of input and output tokens, bounded by quadratic attention and positional coordinate ranges.
Q41	Inference	The KV Cache stores Key and Value activations for past tokens in VRAM to convert sequential decoding compute from $O(L^2)$ to $O(L)$.
Q42	Inference	The first token is slower because the prefill phase processes the entire prompt in parallel, while decoding subsequent tokens is memory-bound.
Q43	Inference	GQA groups query heads to share Key/Value projections, shrinking the KV Cache VRAM footprint to enable larger batch sizes.
Q44	Modern LLMs	BERT is bidirectional for understanding; GPT/Llama are causal decoders for generation; T5 combines both for sequence-to-sequence.
Q45	Modern LLMs	Llama optimizes decoder-only performance using Pre-activation RMSNorm, SwiGLU, RoPE, and GQA.
Q46	Modern LLMs	MoE replaces FFN layers with parallel experts, routing each token to a sparse subset to scale parameter capacity without compute scaling.

Q#	Topic Area	One-Line Takeaway
Q47	Modern LLMs	Router collapse occurs when the routing network over-selects a few experts; we mitigate it using a Load Balancing Auxiliary Loss.
Q48	Reasoning Models	Reasoning models scale Test-Time Compute by generating sequential thinking tokens to explore, verify, and correct logical paths.
Q49	Reasoning Models	Self-correction is an emergent behavior learned via RL, where the model uses its generated error history to redirect attention.
Q50	Reasoning Models	ORMs reward the final output, while PRMs reward every individual reasoning step to prevent correct answers from being reached via faulty logic.
Q51	Limitations	Hallucinations occur because LLMs optimize for token probability over factual truth; we mitigate them using RAG and honesty-oriented RL.
Q52	Evaluation	LLMs are evaluated using perplexity for prediction quality, reasoning benchmarks like GPQA, and human Elo ratings for alignment.
Q53	Production	Inference combines parallel prefill and sequential decode; we optimize it using quantization, GQA, FlashAttention, and PagedAttention.

2. Essential Formula Cheat Sheet

1. Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

2. RMSNorm:

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma \quad \text{where} \quad \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

3. SwiGLU Activation:

$$\text{SwiGLU}(x) = \text{Swish}_1(xW) \otimes (xV)$$

4. ALiBi Attention Score:

$$\text{Score}(q_i, k_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}} - m \cdot |i - j|$$

5. KV Cache VRAM Footprint:

$$\text{VRAM}_{\text{KVCache}} = 4 \cdot B \cdot L \cdot n_{\text{layers}} \cdot n_{\text{heads_kv}} \cdot d_{\text{head}} \text{ Bytes (for 16-bit precision)}$$

6. Chinchilla Compute Cost:

$$C \approx 6ND \text{ FLOPs (Training)} \quad \text{and} \quad C \approx 2N \text{ FLOPs (Inference, per token)}$$

7. Operational Intensity:

$$\text{Intensity} = \frac{\text{FLOPs}}{\text{Memory Access (Bytes)}}$$

8. Perplexity (PPL):

$$\text{PPL}(X) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \ln P(x_i | x_{<i}) \right)$$

3. Top 15 Rapid-Fire Follow-up Q&As

1. Q: Why does self-attention memory scale quadratically?

- **A:** Because comparing every token to every other token generates an $L \times L$ attention matrix that must be stored in GPU memory.

2. Q: What is the primary serving benefit of Grouped-Query Attention (GQA)?

- **A:** It reduces KV Cache memory bandwidth demand, allowing higher batch sizes and throughput.

3. Q: How does FlashAttention achieve faster speeds without reducing FLOPs?

- **A:** By computing attention in blocks in fast GPU Shared Memory (SRAM), avoiding slow High Bandwidth Memory (HBM) read/write bottlenecks.

4. Q: Why does Pre-LN allow training deeper networks than Post-LN?

- **A:** Because Pre-LN keeps the identity path on the residual connection clean, preventing vanishing or exploding gradients.

5. Q: What is exposure bias in language models?

- **A:** The discrepancy where a model is trained always receiving ground-truth tokens (teacher forcing) but must generate using its own predictions at inference.

6. **Q: Why does SwiGLU downscale the intermediate FFN dimension to $\approx \frac{8}{3}d_{\text{model}}$?**
- **A:** To keep the FFN parameter footprint equivalent to standard ReLU FFNs ($8d^2$ weights total) while introducing three projection steps.
7. **Q: What are induction heads?**
- **A:** Attention heads that specialize in copying text patterns (e.g. [A][B] ... [A] -> [B]), which drives in-context learning.
8. **Q: Why do learned positional encodings fail to extrapolate?**
- **A:** Because sequence index positions past the training limit have no learned weights in the embedding table.
9. **Q: What is the advantage of SentencePiece over BPE?**
- **A:** SentencePiece treats whitespace as a character (`▯`) and runs directly on raw bytes, avoiding language-specific pre-tokenizers.
10. **Q: Why is the Decode phase of LLM inference memory-bandwidth bound?**
- **A:** Because it processes only one token at a time, forcing the GPU to spend most of its cycles loading model weights rather than computing.
11. **Q: What is PagedAttention?**
- **A:** An optimization that manages KV Cache memory dynamically in non-contiguous virtual blocks, preventing VRAM fragmentation.
12. **Q: How does DPO align models without a reward model?**
- **A:** It mathematically reformulates the RL objective to optimize the generator directly on preference pairs using a binary cross-entropy loss.
13. **Q: What is the Chinchilla rule of thumb for compute-optimal training?**
- **A:** Training tokens should scale at the same rate as parameter size ($D \approx 20N$).
14. **Q: What is the "needle-in-a-haystack" test?**
- **A:** An evaluation benchmark that tests a model's ability to retrieve a single, hidden fact placed at a random depth in a long context window.
15. **Q: How do reasoning models perform backtracking during inference?**
- **A:** They are trained via RL to output explicit evaluations and self-corrections (e.g. "Wait, this is wrong...") within their generated thinking traces.
-